Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio Aguirre
University of California Santa Cruz

Raquel Barata
University of California Santa Cruz

Raquel Prado
University of California Santa Cruz

Bruno Sansó
University of California Santa Cruz

Abstract

We present the R package **exdqlm** for Bayesian quantile regression, with primary emphasis on dynamic state-space quantile models for time series. The package is built around extended dynamic quantile linear models (exDQLMs), which use the extended asymmetric Laplace (exAL) family, a parametric extension of the asymmetric Laplace (AL) distribution commonly used in quantile regression. The software provides posterior simulation via Markov chain Monte Carlo (MCMC) and fast approximate posterior inference via Laplace–delta variational Bayes (LDVB), supporting posterior uncertainty quantification while also providing a computationally efficient option for longer time series. Furthermore, it supports static exAL quantile regression with regularized priors, dynamic transfer-function models for nonlinear input effects at a given quantile, post hoc posterior-predictive synthesis across separately fitted quantiles, forecasting, and several quantitative and visual diagnostics for model evaluation.

Keywords: dynamic quantile regression, dynamic models, asymmetric Laplace, Laplace–delta variational Bayes, coordinate ascent variational inference, quantile regression.

1. Introduction

Quantile regression has become a widely used alternative to mean-based modeling in static settings. However, dynamic quantile methods and accompanying software remain comparatively limited. In Bayesian parametric quantile regression, the asymmetric Laplace (AL) likelihood has played a central role because it is linked to the check-loss function used in classical quantile regression (Yu and Moyeed 2001; Koenker 2005) and allows convenient mixture

representations for posterior computation. These advantages have made AL-based models attractive, even though the AL distribution can be restrictive as an error model. More flexible error distributions have been studied extensively in the Bayesian nonparametric literature (Kottas and Krnjajić 2009; Reich, Bondell, and Wang 2009; Taddy and Kottas 2010); however, the resulting methods can be computationally demanding for dynamic models and longer time series.

Yan, Zheng, and Kottas (2025) introduce the extended asymmetric Laplace (exAL) distribution, a parametric extension of the asymmetric Laplace distribution that preserves many of its computational advantages while relaxing some of its restrictive assumptions. Building on this family, Barata, Prado, and Sansó (2022) developed the extended dynamic quantile linear model (exDQLM) framework for modeling time-varying quantiles. The **exdqlm** package uses this dynamic model as its foundation, but its software contribution is broader: it provides a unified Bayesian workflow for latent-state quantile dynamics, posterior simulation via Markov chain Monte Carlo (MCMC) and via Laplace–delta variational Bayes (LDVB), dynamic transfer-function modeling, static AL/exAL regression with shrinkage priors, forecasting, diagnostics, and post hoc synthesis across separately fitted quantile models.

The current R (R Core Team 2026) software ecosystem for quantile regression is broad, but the capabilities most relevant to this paper are distributed across software with different aims. Table 1 gives a compact roadmap of representative packages using the distinctions most important for positioning **exdqlm**: modeling scope, temporal or latent-state structure, inference strategy, and handling of multiple quantile levels. To keep the comparison self-contained, the temporal column separates time used only in the regression design from regression autocorrelation, Gaussian latent states, and quantile latent states. The computation column reports how fitting, posterior approximation, or state summaries are obtained, rather than treating these as additional model structures. The purpose of the table is to identify the software gap addressed by the dedicated Bayesian dynamic quantile state-space workflow that **exdqlm** offers.

exdqlm is released under the MIT License and is available from the Comprehensive R Archive Network (CRAN) at `https://CRAN.R-project.org/package=exdqlm`.

The frequentist package **quantreg** (Koenker 2025) remains the broad general-purpose baseline, covering linear, nonlinear, nonparametric, censored, and dynamic quantile regression, together with bootstrap-based and related uncertainty summaries. Its `dynrq()` function supports dynamic linear quantile regression through time-series regression formulas involving lags, differences, trends, seasonal factors, and harmonic terms, while preserving time-series attributes. However, the dynamic structure enters through the design matrix rather than through latent states: once these regressors are constructed, estimation proceeds through the standard **quantreg** fitting routines for a fixed coefficient vector. In that sense, `dynrq()` is a regression-style time-series interface rather than a latent-state state-space model, so it does not provide filtered or smoothed state summaries.

Most other R packages relevant here implement static quantile regression or target a different multi-quantile objective. Among the static Bayesian options, **bayesQR** (Benoit, Al-Hamzawi, Yu, and Van den Poel 2023) provides Bayesian quantile regression under the AL working likelihood with an adaptive-lasso option, whereas **pqrBayes** (Fan, Wu, Ren, Li, and Zhou 2026) emphasizes penalized Bayesian quantile regression with spike-and-slab and horseshoe-family shrinkage priors. The varying-coefficient option, specified by `model = "VC"` in **pqrBayes**,

Package	Main scope	Temporal or state-space handling	Inference or computation	Multiple-quantile handling
quantreg	General frequentist QR	Time-series regression formula: `dynrq()` handles lags, differences, trends, and seasons; no latent states	Optimization-based fitting; frequentist and bootstrap summaries	Vector-valued `tau`; post hoc rearrangement
bayesQR	Bayesian AL QR	Time or lags only as user-supplied covariates; no serial or state-space model	MCMC under the AL likelihood; optional adaptive lasso	Vector-valued quantiles; output by fitted quantile
pqrBayes	Bayesian penalized QR	Varying-coefficient regression: coefficients are B-spline functions of an observed modifier, possibly time; no serial or state-space model	MCMC with shrinkage priors	One target quantile per fit
brms	General Bayesian regression with an AL family	Regression autocorrelation terms can be added; no quantile state-space workflow	Stan/NUTS posterior sampling	Fixed through the `asym_laplace()` quantile parameter
qgam	Smooth additive QR	Time may enter as a smooth covariate; no serial or state-space model	Calibrated Gibbs posterior with GAM fitting	Single quantile via `qgam()`; multiple via `mqgam()`
qrjoint	Bayesian joint linear QR	Time only as a predictor; no serial or state-space model	Adaptive MCMC for joint quantile planes	Joint noncrossing quantile planes
dlm	Gaussian state-space modeling	Gaussian latent states with filtering, smoothing, and forecasting; not QR	Kalman state recursions; MLE or Bayesian parameter analysis	Not a quantile-regression package
exdqlm	Bayesian exAL QR	Quantile latent states with filtering/smoothing summaries, forecasting, and transfer functions	MCMC posterior simulation; VB approximate posterior inference	Post hoc synthesis from separately fitted quantiles

Table 1: Representative R packages relevant to quantile and state-space modeling. Entries summarize documented primary interfaces along the dimensions used to position **exdqlm**; they are selective rather than exhaustive. In the temporal column, time-as-predictor/smooth/modifier uses, regression autocorrelation, Gaussian latent states, and quantile latent states are distinct categories. QR denotes quantile regression, AL asymmetric Laplace, exAL its extended version, NUTS the No-U-Turn sampler, GAM generalized additive model, MLE maximum likelihood estimation, and VB variational Bayes.

models $\gamma_j(V_i)$, where V_i is a user-supplied observed modifier. It builds a B-spline basis in V_i and estimates the basis coefficients by MCMC under shrinkage priors. Thus, **pqrBayes** is appropriate for static varying-coefficient quantile regression, including cases where the chosen modifier is calendar time or another time index. It should not be read as a time-series state-

space method: the coefficient curve is fitted as a smooth regression function of the observed index, not propagated as a latent state from one time point to the next. Consequently, serial dependence, filtering/smoothing, and dynamic forecast propagation are not part of that interface. The broader Bayesian modeling package **brms** (Bürkner 2026) is also relevant because it supports AL-based quantile regression within a flexible multilevel, nonlinear, and additive regression framework, and can incorporate regression-level autocorrelation terms. However, it is not organized as a dedicated quantile state-space workflow, and its documentation does not present filtered or smoothed state summaries of the type targeted by **exdqlm**. Packages such as **qgam** (Fasiolo and Griffiths 2025), **qrjoint** (Tokdar and Cunningham 2025), **quantregGrowth** (Muggeo 2025), and **qrcm** (Frumento 2025) address additive, joint, non-crossing, or coefficient-modeling formulations. Other packages focus on specialized static settings, including penalized frequentist fitting across quantiles in **rqPen** (Sherwood, Li, and Maidman 2026), convolution-smoothed linear quantile regression with efficient gradient-based fitting in **conquer** (He, Pan, Tan, and Zhou 2023), and robust or censored-response modeling in **lqr** (Galarza, Benites, Bourguignon, and Lachos 2024).

Related implementations also exist outside R. In Python, **statsmodels** (statsmodels developers 2025) provides linear quantile regression via iterative reweighted least squares. Julia offers **QuantReg.jl** (Fogarty 2020), and MATLAB includes tools such as `fitrqlinear` and `fitrqnet` (MathWorks 2026). These implementations are useful reference points for the broader software ecosystem, but they are better viewed as general or static quantile-regression tools than as direct counterparts to a dedicated package for Bayesian dynamic quantile state-space modeling.

Viewed against the comparison in Table 1, **exdqlm** is not intended to replace broad general-purpose quantile-regression toolkits, additive quantile-regression packages, or software designed specifically for joint multi-quantile estimation. Its role is more specific: it fills a gap between general quantile-regression software and general Gaussian state-space software by providing a dedicated Bayesian workflow for dynamic quantile state-space models. General state-space packages such as **dlm** (Petris and Gilks 2018) and **dynr** (Ou, Hunter, Chow, Ji, Chen, Hung, Lee, Li, and Park 2021) remain useful for dynamic modeling, but they do not provide the quantile likelihoods, filtered and smoothed quantile-state summaries, diagnostics, forecasting, or synthesis tools targeted here.

The remainder of the paper is organized as follows. In Section 2, we describe the modeling framework, including the algorithms used for posterior inference and transfer-function state augmentation for nonlinear input effects. We also discuss static quantile regression as a special case of the framework. In Section 3 we detail the model diagnostics and forecasting method implemented in the package. Then, in Section 4, we illustrate the package through four examples. Finally, in Section 5, we present a summary of the package capabilities.

2. Extended dynamic quantile linear models

Let y_t denote a scalar response observed at times $t = 1, \ldots, T$. For each t, an extended dynamic quantile linear model (exDQLM) for inference on a single p_0-th quantile is defined by

$$\text{Observation equation:} \quad y_t = \mathbf{F}_t^\top \boldsymbol{\theta}_t + \epsilon_t, \qquad \epsilon_t \sim \text{exAL}_{p_0}(0, \sigma, \gamma) \tag{1}$$

$$\text{System equation:} \quad \boldsymbol{\theta}_t = \mathbf{G}_t \boldsymbol{\theta}_{t-1} + \boldsymbol{\omega}_t, \qquad \boldsymbol{\omega}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{W}_t). \tag{2}$$

Here $\mathbf{F}_t$ is the $q \times 1$ regression vector of the covariates corresponding to the $q \times 1$ parameter vector $\boldsymbol{\theta}_t$ at time t, $\mathbf{G}_t$ is the $q \times q$-dimensional evolution matrix defining the structure of the parameter vector evolution in time, and the normal distribution according to which the system error $\boldsymbol{\omega}_t$ evolves is q-variate. The observational errors ϵ_t of the exDQLM follow an exAL distribution for fixed quantile p_0, whose density is denoted as $\text{exAL}_{p_0}(\cdot)$, and it is such that $\int_{-\infty}^{0} \text{exAL}_{p_0}(\epsilon_t|0,\sigma,\gamma)\text{d}\epsilon_t = p_0$. Thus, the observation equation in Equation (1) implies that $\mathbf{F}_t^\top \boldsymbol{\theta}_t$ corresponds to the p_0-quantile of y_t. Table 2 summarizes how the state-space quantities and prior hyperparameters introduced in this section are represented in the **exdqlm** function inputs.

The exAL distribution, introduced by Yan *et al.* (2025), extends the location-scale mixture representation of the AL (Kozumi and Kobayashi 2011). For fixed p_0, the skewness parameter γ is restricted to an interval (L, U), where L and U are the negative and positive roots of $g(\gamma) = 1 - p_0$ and $g(\gamma) = p_0$, respectively, with $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$. This yields the following mixture representation of $\text{exAL}_{p_0}(0, \sigma, \gamma)$:

$$\begin{aligned}\text{exAL}_{p_0}(y_t \mid 0, \sigma, \gamma) = \int\int_{\mathbb{R}^+\times\mathbb{R}^+} &\mathcal{N}(y_t \mid C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \\ &\times \text{Exp}(v_t \mid \sigma)\mathcal{N}^+(s_t \mid 0, 1)\text{d}v_t\text{d}s_t.\end{aligned} \tag{3}$$

Here, $p = p(p_0, \gamma) = I(\gamma < 0) + \{[p_0 - I(\gamma < 0)]/g(\gamma)\}$ where $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$, $\Phi(\cdot)$ denotes the standard normal CDF, and $I(\cdot)$ is the indicator function. $A(p)$, $B(p)$, $C(p,\gamma)$ are the functions of p and γ: $A(p) = \frac{1-2p}{p(1-p)}$, $B(p) = \frac{2}{p(1-p)}$, $C(p,\gamma) = [I(\gamma > 0) - p]^{-1}$. Lastly, $\text{Exp}(v \mid \sigma)$ denotes the exponential distribution with mean σ, and $\mathcal{N}^+(s_t \mid 0, 1)$ denotes a normal distribution truncated to the positive reals with mean 0 and variance 1. This mixture representation of the exAL can be exploited to rewrite the exDQLM as the following hierarchical model for $t = 1, \ldots, T$:

$$y_t|\boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t \sim \mathcal{N}(y_t \mid \mathbf{F}_t^\top\boldsymbol{\theta}_t + C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \tag{4}$$

$$v_t, s_t|\sigma \sim \text{Exp}(v_t \mid \sigma)\mathcal{N}^+(s_t \mid 0, 1) \tag{5}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t \sim \mathcal{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t). \tag{6}$$

This hierarchical form is leveraged for the posterior inference implemented within the **exdqlm** package.

2.1. Prior specification

Under a Bayesian formulation and given p_0, we specify priors for the initial state vector $\boldsymbol{\theta}_0$, scale parameter σ and skewness parameter γ. For $\boldsymbol{\theta}_0$, we assume a q-variate conjugate normal prior $\boldsymbol{\theta}_0 \sim \mathcal{N}(\mathbf{m}_0, \mathbf{C}_0)$. For σ, we assume a conjugate inverse-gamma prior denoted as $\text{IG}(a_\sigma, b_\sigma)$. In some applications, an informative prior on σ can improve numerical stability and posterior convergence. This is illustrated in the examples of Section 4. The parameter γ has bounded support over the interval (L, U) where L is the negative root of $g(\gamma) = 1 - p_0$ and U is the positive root of $g(\gamma) = p_0$ (Yan *et al.* 2025). The package defaults to a proper Student-t prior on the skewness parameter, truncated to the admissible interval (L, U), which improves numerical stability in the examples below while respecting the support restrictions from the exAL family. Thus, $\gamma \sim \text{t}_{(L,U)}(m_\gamma, s_\gamma)$ with ν_γ degrees of freedom. Table 2 gives the corresponding **exdqlm** input names and default values when applicable.

R input	`model`				`PriorSigma`		`PriorGamma`		
Mathematical symbol	$\mathbf{F}_{1:T}$	$\mathbf{G}_{1:T}$	$\mathbf{m}_0$	$\mathbf{C}_0$	a_σ	b_σ	m_γ	s_γ	ν_γ
R component	`FF`	`GG`	`m0`	`C0`	`a_sig`	`b_sig`	`m_gam`	`s_gam`	`df_gam`
Default if omitted	–	–	$\mathbf{0}_q$	$10^3\mathbf{I}_q$	2.1	1.1	0	1	1

Table 2: Translation from exDQLM notation to **exdqlm** function inputs. The `model` object stores state-space components and the initial-state prior; `PriorSigma` and `PriorGamma` store hyperparameters for σ and γ. A dash indicates no generic default because `FF` and `GG` are determined by the chosen component or supplied model. Here $\mathbf{0}_q$ and $\mathbf{I}_q$ denote a q-vector of zeros and the q-dimensional identity matrix.

2.2. Posterior estimation

The dynamic exDQLM posterior target and the original MCMC and importance-sampling variational Bayes (ISVB) inference strategy were developed by Barata *et al.* (2022). The current package retains that dynamic model family and keeps `exdqlmISVB()` available for historical and backward-compatible workflows. The main variational routine in the current package, however, is `exdqlmLDVB()`, which uses a Laplace–delta approximation for the nonconjugate (σ, γ) block rather than the importance-sampling approximation used by the legacy ISVB path.

The hierarchical representation of the exDQLM facilitates posterior simulation using MCMC algorithms. This is implemented in the function `exdqlmMCMC()`. Gibbs sampling steps are used to update the latent-variable sequences $\{s_t\}_{t=1}^T$ and $\{v_t\}_{t=1}^T$. The dynamic regression coefficients are sampled using a forward-filtering backward-sampling algorithm (Carter and Kohn 1994; Frühwirth-Schnatter 1994). The scale and skewness parameters are updated separately from the latent-state blocks. By default, the package samples σ from its conditional distribution given γ and then updates γ using a bounded univariate slice sampler (Neal 2003); joint random-walk Metropolis–Hastings (MH) alternatives are also available. When desired, the chain can be warm-started from the variational approximation; this initialization is separate from the subsequent MCMC update kernel. An example of this functionality appears in Section 4.1. The routine runs for a total of $N_{\text{BURN}} + N_{\text{MCMC}}$ MCMC iterations, where N_{BURN} is the number of burn-in iterations and N_{MCMC} is the number of retained posterior draws. The object created by `exdqlmMCMC()` includes posterior samples of all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$, filtered and smoothed summaries for the state vector $\boldsymbol{\theta}_{1:T}$, and diagnostics for the proposal kernel used for the (σ, γ) update.

For model comparison or longer time series, posterior sampling with the MCMC algorithm can be computationally demanding. To address this, the package includes a variational routine for fast approximate posterior estimation. The default routine, `exdqlmLDVB()`, uses a mean-field approximation together with a Laplace–delta treatment of the nonconjugate (σ, γ) block. The variational approximation factorizes the posterior into computationally tractable blocks (Ostwald, Kirilina, Starke, and Blankenburg 2014; Beal 2003). Under a mean-field factorization, this induces a blockwise coordinate-ascent variational inference (CAVI) scheme. For the exDQLM, the blocks associated with the dynamic coefficients and latent variables admit recognizable closed-form updates, while the joint variational distribution of σ and γ is nonconjugate. Following the nonconjugate variational strategy of Wang and Blei (2013), we approximate this block on transformed coordinates and use a second-order delta method

to evaluate the expectations of functions of (σ, γ) needed by the remaining variational updates. The dynamic regression coefficients are updated within each iteration using a forward-filtering backward-smoothing algorithm. Convergence is assessed jointly through the state, scale, skewness, and evidence lower bound (ELBO) diagnostics. After the algorithm reaches convergence with tolerance ϵ_{tol}, N_{SAMP} samples are drawn from the variational distributions, resulting in an approximate sample of the posterior distribution; here ϵ_{tol} is the convergence tolerance and N_{SAMP} is the requested number of approximate posterior draws. The output of `exdqlmLDVB()` includes the approximate posterior samples as well as the parameters of the variational distributions for all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$. For the (γ, σ) block, the return also contains the transformed mode, approximate covariance matrix, derived moments, and ELBO terms associated with the Laplace–delta approximation. Selected state-space computations in the dynamic routines are implemented in C++, with corresponding R code retained for parity checks and fallback use. Global package-level runtime options for backend routing and ELBO monitoring are summarized in the supplement. These options are distinct from the function-specific inference inputs listed in Table 3.

Table 3 shows selected core inference controls for the fitting functions `exdqlmMCMC()` and `exdqlmLDVB()`, their mathematical symbols, and their default values. The MCMC routine also accepts `Sig.mh` as an optional proposal covariance when `mh.proposal` selects a joint random-walk MH kernel; it is not used under the default slice-sampling update.

Control	`exdqlmMCMC()`			`exdqlmLDVB()`	
Mathematical symbol	$\mathcal{K}_{\sigma,\gamma}$	N_{BURN}	N_{MCMC}	ϵ_{tol}	N_{SAMP}
R argument	`mh.proposal`	`n.burn`	`n.mcmc`	`tol`	`n.samp`
Default if omitted	`"slice"`	2000	1500	0.1	200

Table 3: Selected core inference controls for `exdqlmMCMC()` and `exdqlmLDVB()`. Here $\mathcal{K}_{\sigma,\gamma}$ denotes the update kernel for the (σ, γ) block, N_{BURN} is the burn-in length, N_{MCMC} is the number of retained MCMC draws, ϵ_{tol} is the LDVB convergence tolerance, and N_{SAMP} is the number of approximate posterior draws. Advanced control-list arguments, optional random-walk MH tuning inputs, and package-level backend options are documented separately.

2.3. Specification of the evolution covariance via discount factors

Both estimation algorithms, `exdqlmMCMC()` and `exdqlmLDVB()`, use discount factors to specify the time-evolving covariance matrices $\mathbf{W}_t$ introduced in Equation (2) (see West and Harrison 1997; Prado, Ferreira, and West 2021, and references therein). More precisely, we define the evolution variance matrix $\mathbf{W}_t = \frac{1-\delta}{\delta}\mathbf{G}_t\mathbf{C}_{t-1}\mathbf{G}_t^\top$ where $\mathbf{C}_{t-1}$ denotes the posterior variance of the state vector $\boldsymbol{\theta}_{t-1}$ at time $t-1$ and δ is a discount factor such that $0 < \delta \leq 1$. The discount factor δ can be specified by the user via function parameter `df`. Selection of discount factors is typically done by optimizing a model-checking criterion. In practice we recommend predictive criteria such as the continuous ranked probability score (CRPS) or posterior predictive loss criterion (PPLC) defined in Section 3, with Kullback–Leibler (KL) divergence as a complementary calibration diagnostic. A brief example of this method for discount factor selection can be found in Section 4.2.

The routines also support component-specific discounting. Suppose the state vector is comprised of h components $\boldsymbol{\theta}_{it}$ of dimension q_i for $i = 1, \ldots, h$ such that $\boldsymbol{\theta}_t^\top = (\boldsymbol{\theta}_{1t}^\top, \ldots, \boldsymbol{\theta}_{ht}^\top)$ and

$\sum_{i=1}^{h} q_i = q$. If $\mathbf{W}_{it}$ denotes the i^{th} component of the evolution covariance matrix such that $\mathbf{W}_t = \text{blockdiag}\{\mathbf{W}_{1t}, \ldots, \mathbf{W}_{ht}\}$ and δ_i denotes a discount factor for the i^{th} component, we can define the evolution variance matrix by component as $\mathbf{W}_{it} = \frac{1-\delta_i}{\delta_i}\mathbf{G}_{it}\mathbf{C}_{i,t-1}\mathbf{G}_{it}^{\top}$. Here $\mathbf{G}_{it}$ and $\mathbf{C}_{i,t-1}$ denote the i^{th} components of $\mathbf{G}_t$ and $\mathbf{C}_{t-1}$, respectively. The function parameters `df` and `dim.df` allow users to specify a set of component discount factors $(\delta_1, \ldots, \delta_h)$ and their dimensions $(q_1, \ldots, q_h)$, respectively, within the state-space model. Examples of this feature can be found in Section 4. For a full review of discount factors including discounting by component, see West and Harrison (1997).

2.4. Transfer function model

Transfer functions provide a state-space representation for current and lagged input effects on a response quantile. In mean-centric dynamic modeling, transfer functions are a standard method to incorporate variables that measure the combined effect of current and past regression effects (West and Harrison 1997). Within **exdqlm**, this extension is handled by augmenting the dynamic state-space representation conditional on a fixed transfer-function rate λ, and the resulting workflow is available through both LDVB and MCMC fitting routines.

For $t = 1, \ldots, T$ and transfer input vector $\mathbf{x}_t \in \mathbb{R}^k$, a transfer-function exDQLM with exponential decay is defined by

$$y_t|\boldsymbol{\theta}_t, \zeta_t, \gamma, \sigma \sim \text{exAL}_{p_0}(\mathbf{F}_t^{\top}\boldsymbol{\theta}_t + \zeta_t, \sigma, \gamma) \tag{7}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t^{\theta} \sim \mathcal{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t^{\theta}) \tag{8}$$

$$\zeta_t|\zeta_{t-1}, \boldsymbol{\psi}_{t-1}, \omega_t \sim \mathcal{N}(\lambda\zeta_{t-1} + \mathbf{x}_t^{\top}\boldsymbol{\psi}_{t-1}, \omega_t) \tag{9}$$

$$\boldsymbol{\psi}_t|\boldsymbol{\psi}_{t-1}, \mathbf{W}_t^{\psi} \sim \mathcal{N}(\boldsymbol{\psi}_{t-1}, \mathbf{W}_t^{\psi}). \tag{10}$$

Here $\zeta_t \in \mathbb{R}$ denotes the accumulated transfer effect on the quantile at time t, while $\boldsymbol{\psi}_t \in \mathbb{R}^k$ is the vector of instantaneous transfer coefficients associated with the components of $\mathbf{x}_t$. The mean contribution of the transfer block is therefore $\lambda\zeta_{t-1} + \mathbf{x}_t^{\top}\boldsymbol{\psi}_{t-1}$: the inner product $\mathbf{x}_t^{\top}\boldsymbol{\psi}_{t-1}$ captures the contemporaneous effect of the current inputs, and the parameter $\lambda \in (0, 1)$ controls how much of the previous accumulated effect is propagated forward. For $0 < \lambda < 1$, this propagated contribution decays geometrically. More explicitly, the contribution of $\mathbf{x}_t$ to the quantile at time $t + h$ is $\lambda^h a_t$, where $a_t = \mathbf{x}_t^{\top}\boldsymbol{\psi}_{t-1}$. For tolerance $\epsilon > 0$, the package summarizes the continuous decay horizon

$$k_t = \begin{cases} \{\log(\epsilon) - \log(|a_t|)\}/\log(\lambda), & |a_t| > \epsilon, \\ 0, & |a_t| \leq \epsilon, \end{cases}$$

whose positive values solve $\lambda^{k_t}|a_t| = \epsilon$. The reported `median.kt` is the median of these horizons over time for $\epsilon = 1\text{e}{-3}$.

Conditional on a fixed λ, say $\tilde{\lambda}$, we augment the dynamic fitting routines to incorporate the transfer-function structure by replacing $\mathbf{F}_t$, $\boldsymbol{\theta}_t$, $\mathbf{G}_t$, and $\mathbf{W}_t^{\theta}$ in Equations (1)-(2) with

$$\tilde{\mathbf{F}}_t^{\top} = (\mathbf{F}_t^{\top}, 1, \mathbf{0}_k^{\top}), \qquad \tilde{\boldsymbol{\theta}}_t^{\top} = (\boldsymbol{\theta}_t^{\top}, \zeta_t, \boldsymbol{\psi}_t^{\top}),$$

$$\tilde{\mathbf{G}}_t = \text{blockdiag}\{\mathbf{G}_t, \mathbf{G}_t^{tf}\}, \qquad \mathbf{G}_t^{tf} = \begin{pmatrix} \tilde{\lambda} & \mathbf{x}_t^{\top} \\ \mathbf{0}_k & \mathbf{I}_k \end{pmatrix},$$

and

$$\tilde{\mathbf{W}}_t = \text{blockdiag}\{\mathbf{W}_t^{\theta}, \omega_t, \mathbf{W}_t^{\psi}\}.$$

This augmentation, which extends the routines found in `exdqlmLDVB()` and `exdqlmMCMC()`, is implemented in the functions `exdqlmTransferLDVB()` and `exdqlmTransferMCMC()`.

Similar to the specification of $\mathbf{W}_t^{\theta}$ discussed in Section 2.3, discount factors are used to specify the transfer block variances ω_t and $\mathbf{W}_t^{\psi}$. In the package, `tf.df` can be supplied as a single shared discount factor, as a pair $(\delta_\zeta, \boldsymbol{\delta}_{\boldsymbol{\psi}})$ with a shared value for the whole $\boldsymbol{\psi}_t$ block, or componentwise across $(\zeta_t, \psi_{1,t}, \ldots, \psi_{k,t})$. A conjugate normal prior on $(\zeta_0, \boldsymbol{\psi}_0^\top)^\top \sim \mathcal{N}(\mathbf{m}_0^{tf}, \mathbf{C}_0^{tf})$, with $\mathbf{m}_0^{tf} \in \mathbb{R}^{k+1}$ and $\mathbf{C}_0^{tf} \in \mathbb{R}^{(k+1)\times(k+1)}$, completes the transfer-function model extension. Table 4 shows the additional transfer-function inputs, their mathematical symbols, and their accepted forms or defaults in the package interface.

Quantity	Mathematical symbol	R argument	Input/default
Transfer inputs	$\mathbf{x}_t$	`X`	Required numeric vector or $T \times k$ matrix.
Transfer rate	$\tilde{\lambda}$	`lam`	Required scalar in $(0, 1)$.
Discount factors	$(\delta_\zeta, \boldsymbol{\delta}_{\boldsymbol{\psi}})$	`tf.df`	Required vector: length 1 (shared); length 2 (δ_ζ and shared $\boldsymbol{\delta}_{\boldsymbol{\psi}}$); or length $k+1$ (componentwise).
Prior mean	$\mathbf{m}_0^{tf}$	`tf.m0`	Optional vector of length $k+1$; default $\mathbf{0}_{k+1}$.
Prior covariance	$\mathbf{C}_0^{tf}$	`tf.C0`	Optional $(k+1) \times (k+1)$ covariance matrix; default $\mathbf{I}_{k+1}$.

Table 4: Transfer-function inputs for `exdqlmTransferLDVB()` and `exdqlmTransferMCMC()`. Here k denotes the number of transfer covariates.

2.5. Special cases, including static quantile regression

The first special case follows from the fact that the exAL is an extension of the AL (Yan *et al.* 2025). When $\gamma = 0$, the observational error ϵ_t in Equation (1) reduces such that $\epsilon_t \sim \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma) = \text{AL}_{p_0}(\epsilon_t|0, \sigma)$. This special case with observational errors distributed independently from an AL is the dynamic quantile linear model (DQLM) presented in Gonçalves, Migon, and Bastos (2017). The DQLM can be implemented with our package using the parameter `dqlm.ind = TRUE`.

Non-time-varying quantile regression is also a special case of the same exAL modeling framework. With identity evolution, $\mathbf{G}_t = \mathbf{I}$, and no state innovation, $\mathbf{W}_t = \mathbf{0}$, the coefficients are time-invariant and the model reduces to the static exAL regression methodology of Yan *et al.* (2025). The static exAL special case is implemented directly in the package. This model can be implemented using our functions `exalStaticMCMC()` or `exalStaticLDVB()`, which provide MCMC and LDVB algorithms, respectively.

The final special case is the intersection of the previous two. A static quantile regression model with observational errors distributed according to an AL corresponds to Bayesian linear quantile regression under the AL working likelihood, first presented in Yu and Moyeed (2001). This methodology, utilized to create the Bayesian package for quantile regression **bayesQR**, can also be implemented with our package using functions `exalStaticMCMC()` or `exalStaticLDVB()` with parameter `al.ind = TRUE` (a static convenience alias of `dqlm.ind = TRUE`), which fixes $\gamma = 0$.

Examples of these dynamic and static special cases appear in Section 4, with the sparse static regression workflow illustrated in Section 4.4.

3. Model diagnostics and forecasting

To evaluate the quantile inference and predictive performance of the exDQLM, several quantitative and visual model diagnostics are included in the package.

The one-step-ahead predictive probability integral transform (PIT) introduced by Rosenblatt (1952) is a useful diagnostic for models in which the marginal forecast distributions are unrecognizable or difficult to compute (Huerta, Jiang, and Tanner 2003; Prado, Molina, and Huerta 2006). The ideal one-step-ahead PIT is

$$u_t^\star = \Pr(Y_t \leq y_t \mid y_{1:t-1}), \tag{11}$$

where $y_{1:t-1}$ denotes the observations available before time t. Under the correct one-step-ahead predictive distribution, the PIT values in (11) are independent Uniform$(0,1)$ variates (Rosenblatt 1952). The package reports a maximum a posteriori (MAP) plug-in version based on the conditional Gaussian filtering representation. If $\hat{f}_t$ and $\hat{q}_t$ denote the MAP plug-in one-step-ahead forecast location and variance, the stored standardized forecast error is

$$\hat{e}_t = \frac{y_t - \hat{f}_t}{\sqrt{\hat{q}_t}}, \tag{12}$$

and the plug-in PIT diagnostic is

$$\hat{u}_t = \Phi(\hat{e}_t), \tag{13}$$

where Φ denotes the standard normal CDF. These plug-in quantities are diagnostic summaries of the fitted one-step-ahead sequence rather than posterior uncertainty bands for residuals.

Using this estimated sequence $\{\hat{u}_t\}$, we can assess the model performance in several ways. First, we visually examine the correlation of the estimated sequence via an autocorrelation function (ACF) plot. Second, by transforming the values with a standard normal inverse CDF, we compare their distributional shape to that of a standard normal with a normal quantile–quantile (QQ) plot. To quantify the divergence of this transformed sequence from normality we use the KL divergence (Kullback and Leibler 1951), $\mathrm{KL}(h,\phi) = \int_{-\infty}^{\infty} h(x) \log \frac{h(x)}{\phi(x)} \mathrm{d}x$, where h denotes the diagnostic sample distribution and ϕ is the standard normal density. The package reports a deterministic one-dimensional KL normality diagnostic: the forward value estimates $\mathrm{KL}(P_e \,||\, N(0,1))$ for the MAP standardized forecast-error distribution P_e, and the flipped value estimates the reversed diagnostic $\mathrm{KL}(N(0,1) \,||\, P_e)$. The forward calculation uses the semiclosed identity based on analytic normal cross-entropy and a one-dimensional entropy estimate; the flipped calculation is reported as a secondary sensitivity diagnostic. Smaller values suggest better calibration. The MAP standardized forecast errors in (12) can also be used as an additional graphical aid in examining the distribution at specific time points. These MAP standardized forecast errors are included in the output of the `exdqlmMCMC()` and `exdqlmLDVB()` functions.

In addition to the calibration-based diagnostics above, we compute the mean continuous ranked probability score (CRPS) from the posterior predictive distribution (Gneiting and

Raftery 2007). Let $\rho_p(x) = x[p - I(x < 0)]$ denote the check loss at level p. For a predictive distribution G with quantile function G^{-1} and an observation y, the CRPS is

$$\text{CRPS}(y, G) = \mathbb{E}_{Y\sim G}|Y - y| - \frac{1}{2}\mathbb{E}_{Y,Y'\sim G}|Y - Y'| = 2\int_0^1 \rho_p\Big(y - G^{-1}(p)\Big)\, dp, \tag{14}$$

which shows that CRPS integrates check loss across all quantile levels and therefore evaluates the full predictive distribution rather than a single target level p_0. In `exdqlmDiagnostics()`, the posterior predictive draws define empirical quantiles $\widehat{q}_t(\tau_k)$, and the reported pointwise score is the finite integrated quantile-score approximation

$$\widehat{\text{CRPS}}_t = 2\sum_{k=1}^{K} w_k \rho_{\tau_k}\{y_t^{obs} - \widehat{q}_t(\tau_k)\}, \tag{15}$$

where the weights w_k sum to one (Laio and Tamea 2007). The package default uses the grid $\tau_k = 0.01, 0.02, \ldots, 0.99$ with equal weights, and the reported mean CRPS averages (15) over time. Smaller CRPS values indicate better calibrated and sharper predictive performance. For deterministic forecasts, CRPS reduces to the absolute error, so the sample mean CRPS equals the MAE.

For a final diagnostic, we include the Gelfand and Ghosh (1998) posterior predictive loss criterion (PPLC), returned by the package as `pplc`, which evaluates the posterior predictive distribution under the target-level check loss ρ_{p_0}. That is,

$$\text{PPLC} = \sum_t \mathsf{E}[\rho_{p_0}(y_t^{obs} - y_t^{rep}) \mid y_{1:T}], \tag{16}$$

where the expectation is with respect to posterior predictive replicate draws generated from the observation equation (1) under posterior draws of the model parameters. Thus, CRPS summarizes the full predictive distribution through the integrated quantile-score approximation in (15), whereas PPLC keeps the target quantile explicit through ρ_{p_0}. Smaller PPLC values indicate better performance under this target-quantile loss criterion. Samples from the posterior replicate distributions are included in the output of the `exdqlmMCMC()` and `exdqlmLDVB()` functions.

The function `exdqlmDiagnostics()` implements the model checking criteria discussed in this section. When an output from either `exdqlmMCMC()` or `exdqlmLDVB()` is passed to `exdqlmDiagnostics()`, the resulting object includes: the estimated sequence $\{\hat{u}_t\}$, the KL divergence and flipped KL divergence, the mean CRPS, `pplc`, the ordered pairs of the QQ-plot, and autocorrelations by lag. By default, the function also produces the QQ-plot, ACF plot, and MAP standardized forecast-error plot. Examples of the utility of `exdqlmDiagnostics()` can be found in Section 4.

3.1. k-step-ahead forecast

For an arbitrary time $\tilde{t}$, the k-step-ahead future marginal distribution of the quantile is

$$\mathbf{F}_{\tilde{t}+k}^\top \boldsymbol{\theta}_{\tilde{t}+k} \mid y_1, \ldots, y_{\tilde{t}} \sim \mathcal{N}(\mathbf{F}_{\tilde{t}+k}^\top \mathbf{a}_{\tilde{t}}(k), \mathbf{F}_{\tilde{t}+k}^\top \mathbf{R}_{\tilde{t}}(k)\mathbf{F}_{\tilde{t}+k}) \tag{17}$$

where $\mathbf{a}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{a}_{\tilde{t}}(k-1)$, $\mathbf{R}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k-1)\mathbf{G}_{\tilde{t}+k}^\top + \mathbf{W}_{\tilde{t}+k}$, $\mathbf{a}_{\tilde{t}}(0) = \mathbf{m}_{\tilde{t}}$, and $\mathbf{R}_{\tilde{t}}(0) = \mathbf{C}_{\tilde{t}}$, with $\mathbf{m}_{\tilde{t}}$ and $\mathbf{C}_{\tilde{t}}$ denoting the filtered mean and covariance of $\boldsymbol{\theta}_{\tilde{t}}$, respectively. This

distribution is implemented in the function `exdqlmForecast()`, which is compatible with the outputs from both the MCMC and LDVB algorithms. The fitted object supplies the filtered posterior summaries at the forecast origin, while optional future observation and evolution matrices are supplied when the required forecast design is not already stored in the fitted model. Table 5 summarizes the forecast inputs and optional posterior predictive draw controls. The output of `exdqlmForecast()` includes the parameters of the forecasted distribution as well as $\mathbf{a}_{\tilde{t}}(k)$ and $\mathbf{R}_{\tilde{t}}(k)$ for the k steps, with optional posterior predictive draw matrices available when requested.

Quantity	Mathematical symbol	R argument	Input/default
Fitted dynamic model	$(\mathbf{m}_{\tilde{t}}, \mathbf{C}_{\tilde{t}})$	`m1`	Required fitted dynamic object from `exdqlmMCMC()`, `exdqlmLDVB()`, or legacy `exdqlmISVB()`.
Forecast origin	$\tilde{t}$	`start.t`	Required integer time index within the fitted model.
Forecast horizon	k	`k`	Required positive integer number of forecast steps.
Future observation vectors	$\mathbf{F}_{(\tilde{t}+1):(\tilde{t}+k)}$	`fFF`	Optional $q \times 1$ or $q \times k$ matrix; required with `fGG` when forecasting beyond the stored model matrices.
Future evolution matrices	$\mathbf{G}_{(\tilde{t}+1):(\tilde{t}+k)}$	`fGG`	Optional $q \times q$ matrix or $q \times q \times k$ array; required with `fFF` when forecasting beyond the stored model matrices.
Credible interval mass	$1-\alpha$	`cr.percent`	Optional scalar in $(0,1)$; default `0.95`.
Forecast draws	$Y^{rep}_{\tilde{t}+1:\tilde{t}+k}$	`return.draws`; `n.samp`; `seed`	Optional controls; default `return.draws = FALSE`. When requested, `n.samp` sets the number of draw columns and `seed` makes generation reproducible.

Table 5: Forecast notation and main `exdqlmForecast()` inputs. Here q denotes the state dimension.

4. Examples

We demonstrate the general workflow and key features of the package through three real-data analyses and one simulation benchmark. The first three examples illustrate dynamic workflows, while the fourth illustrates the static exAL special case. Table 6 provides an overview of the **exdqlm** functions and the resulting objects. The functions in Table 6 cover the high-level modeling workflow used in the examples. The package also exposes distribution-level utilities for the exAL error family: `dexal()`, `pexal()`, `qexal()`, and `rexal()` evaluate the density, distribution function, quantile function, and random generator, respectively, and `get_gamma_bounds()` returns the admissible interval for γ at a specified p_0. These utilities are useful for simulation, prior calibration, and direct checks of the exAL distribution, but are not needed for the main fitting, forecasting, diagnostic, or synthesis workflows illustrated below. We use CrI as shorthand for credible interval in figure captions and plotting labels. All figures and tables are generated from scripts distributed with the article repository. The top-level `code.R` file is the canonical reproduction entry point, and the scripts under `analysis/manuscript/examples/` generate the displayed figures, tables, logs, and

Function	Returned object	Role in the workflow
Dynamic model construction		
`polytrendMod()`	`exdqlm`	Create polynomial trend component.
`seasMod()`	`exdqlm`	Create Fourier-form seasonal or periodic component.
`regMod()`	`exdqlm`	Create regression component from covariates.
`as.exdqlm()`	`exdqlm`	Coerce a compatible list or time-invariant `dlm` object.
Dynamic model fitting – Input `exdqlm` objects		
`exdqlmMCMC()`	`exdqlmMCMC`	Fit dynamic exDQLM/DQLM via MCMC.
`exdqlmLDVB()`	`exdqlmLDVB`	Fit dynamic exDQLM/DQLM via LDVB.
`exdqlmTransferLDVB()`	`exdqlmLDVB`	Fit transfer-function exDQLM via LDVB.
`exdqlmTransferMCMC()`	`exdqlmMCMC`	Fit transfer-function exDQLM via MCMC.
Static model fitting – Input design matrix and response		
`exalStaticLDVB()`	`exalStaticLDVB`	Fit static exAL/AL regression via LDVB.
`exalStaticMCMC()`	`exalStaticMCMC`	Fit static exAL/AL regression via MCMC.
Dynamic fit examination – Input `exdqlmMCMC`, `exdqlmLDVB`, or legacy `exdqlmISVB` objects		
`exdqlmForecast()`	`exdqlmForecast`	Compute forecast quantiles and optional predictive draws.
`exdqlmDiagnostics()`	`exdqlmDiagnostic`	Compute calibration and predictive diagnostics; optionally plot them.
`exdqlmPlot()`	Invisible `list`	Plot dynamic quantile MAP and CrI summaries.
`compPlot()`	Invisible `list`	Plot component or state-level MAP and CrI summaries.
Static fit examination – Input `exalStaticMCMC` or `exalStaticLDVB` objects		
`exalStaticDiagnostics()`	`exalStaticDiagnostic`	Compute static fitted-quantile diagnostics and coefficient-interval summaries/plots.
Predictive synthesis – Input dynamic fit/forecast objects or draw matrices		
`quantileSynthesis()`	`exdqlmSynthesis`	Synthesize posterior predictive draws across separately fitted quantile levels.

Table 6: Core workflow functions supporting the examples. Fitting, diagnostic, forecast, static, and synthesis routines return the objects shown using R's S3 object system; `exdqlmPlot()` and `compPlot()` return invisible summary lists.

provenance files. The code chunks shown below are compact, reader-facing excerpts of those scripts; generic output handling, caching, graphics devices, and manifest-writing code are kept in the replication scripts. The accompanying reproducibility materials record the computing environment and outputs used in the paper.

We begin by loading the **exdqlm** package used for all examples.

```
R> library("exdqlm")
```

4.1. Lake Huron

In this example, we consider the Lake Huron time series from the package **datasets**. The data are annual measurements of the level (ft) of Lake Huron from 1875 to 1972, shown in Figure

2. This example shows how to build a basic state-space structure, specify a prior on γ, run the MCMC algorithm, and plot estimated quantiles and forecast distributions.

To estimate the dynamic distribution of the data, we consider three quantiles, $p_0 = 0.95, 0.50,$ and 0.05. We begin by creating the state-space structure and prior used to estimate the quantiles (i.e. $\mathbf{F}_t, \mathbf{G}_t, \mathbf{m}_0$, and $\mathbf{C}_0$ discussed in Section 2). We choose to model the quantiles with a second order polynomial trend, which we construct with the `polytrendMod()` function. The initial level prior is centered at 579 ft, a rounded domain-scale value chosen before fitting rather than the sample mean, and the initial slope prior is centered at zero.

```
R> model = polytrendMod(order = 2, m0 = c(579, 0),
+                       C0 = 10 * diag(2))
R> model

Prior mean of the state vector:
$m0
         [,1]
[1,] 579.0000
[2,]   0.0000


Prior covariance of the state vector:
$C0
     [,1] [,2]
[1,]   10    0
[2,]    0   10


Observational vector:
$FF
     [,1]
[1,]    1
[2,]    0


Evolution matrix:
$GG
     [,1] [,2]
[1,]    1    1
[2,]    0    1
```

To allow variability in the dynamic quantile, we select a single discount factor of 0.9 to define the evolution covariance matrix $\mathbf{W}_t$. Discount factors can also be optimized, an example of which we will show in Section 4.2. We place truncated Student-t priors on the skewness parameters via `PriorGamma` as discussed in Section 2.1. The prior centers for γ are quantile-specific: they are centered at zero for the median and shifted in opposite directions for the upper and lower tails to provide stable tail-specific initialization within the admissible support. In the fits shown here we use `n.burn = 2000` and `n.mcmc = 3000`. For the final fit shown below ($p_0 = 0.05$), we display R progress updates every 1000 iterations using `verbose = TRUE` and `verbose.every = 1000`. The progress output also reports which computational backend is being used. In the current implementation, some operations, including dynamic MCMC updates and sampling from latent-variable full conditionals, can be routed through either R or C++ implementations. The package-level options controlling this routing are listed in the supplementary backend-options table.

```
R> set.seed(20260501)
R> M95 = exdqlmMCMC(y = LakeHuron, p0 = 0.95, model = model,
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = -1, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                    df = 0.9, dim.df = 2,
+                    PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                    n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M5 = exdqlmMCMC(y = LakeHuron, p0 = 0.05, model = model,
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = 1, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = TRUE, verbose.every = 1000)


MCMC backend: C++ (mode=fast)
running LDVB algorithm to initialize mcmc
GIG backend: C++ Devroye (required)
MCMC start | burn=2000.00 | keep=3000.00 | kernel=slice | warm_start=ldvb
MCMC progress | model=exDQLM | phase=burn | iter=1000/5000 | kept=0/3000
MCMC progress | model=exDQLM | phase=burn | iter=2000/5000 | kept=0/3000
MCMC progress | model=exDQLM | phase=keep | iter=3000/5000 | kept=1000/3000
MCMC progress | model=exDQLM | phase=keep | iter=4000/5000 | kept=2000/3000
MCMC progress | model=exDQLM | phase=keep | iter=5000/5000 | kept=3000/3000
MCMC done | model=exDQLM | status=complete | iter=5000.00 | runtime_sec=16.272
```

By default, `exdqlmMCMC()` uses the package's LDVB routine, `exdqlmLDVB()`, to warm-start the chain. This initialization choice is separate from the subsequent MCMC kernel. Because the data are relatively symmetric, we do not expect the skewness parameter γ to be far from zero at $p_0 = 0.5$. Consistent with this, the posterior mean of γ is -0.012 with a 95% credible interval $(-0.429, 0.462)$, while the posterior mean of σ is 0.369 with a 95% credible interval $(0.282, 0.468)$. All retained samples in the output of `exdqlmMCMC()` are `mcmc` objects, so we can use the package **coda** (Plummer, Best, Cowles, and Vines 2006) to examine the trace and density plots of both σ and γ. For readability, we plot every 10th retained draw in Figure 1.

```
R> par(mfcol = c(2, 2), mar = c(4.1, 4.1, 2.1, 1.0))
R> keep.idx = seq(1, length(M50$samp.sigma), by = 10)
R> sigma.trace = coda::mcmc(M50$samp.sigma[keep.idx], thin = 10)
R> gamma.trace = coda::mcmc(M50$samp.gamma[keep.idx], thin = 10)
R> coda::traceplot(sigma.trace, main = "sigma trace")
R> coda::densplot(sigma.trace, main = "sigma density")
R> coda::traceplot(gamma.trace, main = "gamma trace")
R> coda::densplot(gamma.trace, main = "gamma density")
```

The posterior summary does not provide strong evidence that γ differs from zero for the median fit. We therefore re-run the model with a point-mass prior on γ at 0 using the settings `gam.init = 0` and `fix.gamma = TRUE`, which is equivalent to the setting `dqlm.ind = TRUE`. This reduces the model to the DQLM mentioned in Section 2.5.

```
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                    df = 0.9, dim.df = 2,
+                    gam.init = 0, fix.gamma = TRUE,
+                    n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
```

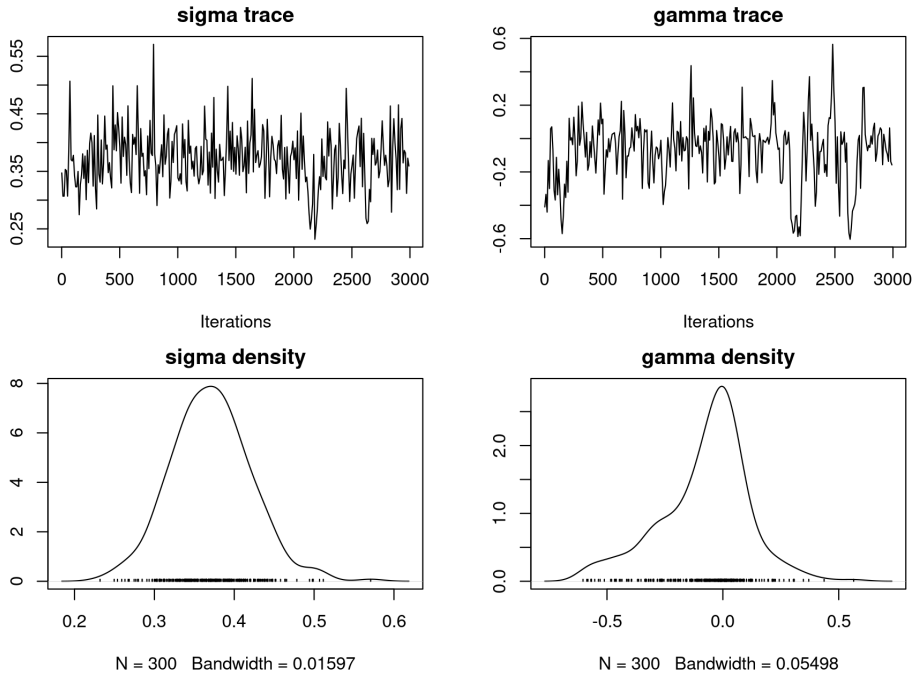

Figure 1: Example 1: Lake Huron. Trace and density plots from the median run with 2000 burn-in iterations and 3000 retained draws. For readability, every 10th retained draw is used in the plotting step. The top row shows the posterior trace plots for σ and γ, and the bottom row shows the corresponding posterior densities. The displayed paths are representative MCMC traces from the recorded run profile; exact trace paths may differ across reruns while retaining comparable posterior summaries.

The results are `exdqlmMCMC` objects that can be used for analysis and forecasting. First we examine the estimated quantiles with the plot method for `exdqlmMCMC` objects, seen in the top-left panel of Figure 2. This generates a plot of the data with the MAP estimates and the 95% CrIs of the dynamic quantile. Subsequent quantiles (i.e. $p_0 = 0.50, 0.05$ in this example) can be added to the plot using the function `exdqlmPlot()` with the argument `add = TRUE`.

```
R> par(mfrow = c(2, 2), mar = c(4.4, 4.1, 2.2, 1.2),
+        oma = c(0, 0, 0.8, 0))
R> plot(M95); title("(a) Dynamic quantiles")
R> exdqlmPlot(M50, add = TRUE, col = "blue")
R> exdqlmPlot(M5, add = TRUE, col = "forest green")
R> legend("topright", lty = 1, bty = "n",
+        col = c("purple", "blue", "forest green"),
+        legend = c(expression('p'[0]*'=0.95'), expression('p'[0]*'=0.50'),
+                   expression('p'[0]*'=0.05')))
```

Next, we forecast the $k = 8$ step-ahead distributions starting in 1972. To do this, we first create the observational vector (`fFF`) and evolution matrix (`fGG`) that will be used in the forecast updates. In this case both are non-time-varying and identical to those used to estimate the quantile.

```
R> fFF = model$FF
```

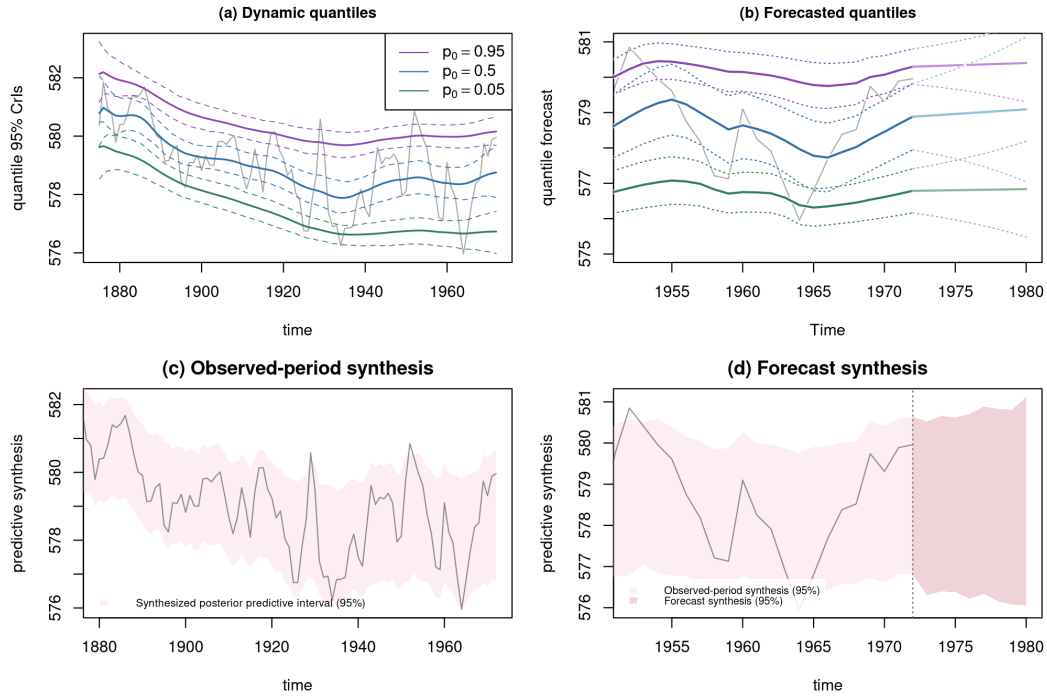

Figure 2: Example 1: Lake Huron. Top-left: MAP estimates and 95% CrIs of the estimated quantiles, plotted with the data (grey). Top-right: forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (seen from 1952 to 1972). Bottom-left: synthesized posterior predictive 95% interval over the observed period. Bottom-right: observed-period synthesis near the forecast origin (lighter band) and synthesized posterior predictive 95% interval over the eight-step forecast window (slightly darker rose band); the vertical dotted line marks the forecast origin.

```
R> fGG =  model$GG
```

We plot the time series data in a narrower range for closer examination and add the forecast estimates using `exdqlmForecast()` with argument `add = TRUE`. Notice we start the forecast at the last time index of the data, i.e. `start.t = length(LakeHuron)`.

```
R> plot(LakeHuron,  xlim = c(1952, 1980), ylim = c(575, 582),
+        col = "dark grey", main = "(b) Forecasted quantiles",
+           ylab = "forecast 95% CrIs")
R> fc95 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M95,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               return.draws = TRUE)
R> fc50 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M50,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("blue", "light blue"), return.draws = TRUE)
R> fc05 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M5,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("forest green", "green"), return.draws = TRUE)
```

This generates a plot of the data with the forecasted quantile estimates and 95% credible

intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (before 1972), for reference. Results can be seen in the top-right panel of Figure 2. The percentage in the CrIs can be modified with the parameter `cr.percent`.

When several quantile models are fitted separately, the package also allows their posterior predictive draws to be combined into a single coherent predictive distribution through `quantileSynthesis()`. Each fitted model contributes local predictive information near its target quantile, and the synthesis step interpolates across these quantile-specific draws to obtain one posterior predictive distribution. When independently fitted quantiles induce crossing, optional monotonicity corrections based on isotonic adjustment and monotone rearrangement are available (Barlow, Bartholomew, Bremner, and Brunk 1972; Chernozhukov, Fernández-Val, and Galichon 2010). For the Lake Huron example, we apply this routine directly to the fitted objects `M5`, `M50`, and `M95` over the observed period.

```
R> syn.obs = quantileSynthesis(
+               draws_list = list(M5, M50, M95),
+               p = c(0.05, 0.50, 0.95),
+               T_expected = length(LakeHuron))
```

Because the argument `return.draws = TRUE` was used when producing the forecast objects (`fc05`, `fc50`, `fc95`) above, each object stores posterior predictive forecast draws in the return value `samp.fore`. This allows us to apply the same synthesis step to the eight-step forecast horizon.

```
R> syn.fore = quantileSynthesis(
+               draws_list = list(fc05, fc50, fc95),
+               p = c(0.05, 0.50, 0.95),
+               T_expected = 8)
```

The objects `syn.obs` and `syn.fore` have class `exdqlmSynthesis`, so the corresponding `plot()` method can be used to display the synthesized posterior predictive intervals. The lower panels of Figure 2 are produced from these objects as follows. The first synthesis panel shows the observed-period synthesized predictive interval, and the second panel uses the same observed-period synthesis near the forecast origin, overlays the forecast synthesis, and shades the one-step bridge from the final observed synthesis interval to the first forecast synthesis interval.

```
R> synth.obs.col = adjustcolor("#F7D6DE", alpha.f = 0.40)
R> synth.fore.col = adjustcolor("#D98A9B", alpha.f = 0.38)
R> plot(syn.obs, y = LakeHuron, time = as.numeric(time(LakeHuron)),
+         xlim = c(1880, 1972), ylim = c(575.75,582.25),
+         ylab = "predictive synthesis",
+         main = "(c) Observed-period synthesis",
+         show.median = FALSE, band.col = synth.obs.col,
+         y.col = adjustcolor("grey30", alpha.f = 0.62))
R> legend("bottomleft",
+           legend = "Synthesized posterior predictive interval (95%)",
+           fill = synth.obs.col, border = NA, bty = "n", cex = 0.68)
R> plot(syn.obs, y = LakeHuron, time = as.numeric(time(LakeHuron)),
+         xlim = c(1952, 1980), ylim = c(575.75,581),
+         ylab = "predictive synthesis",
+         main = "(d) Forecast synthesis",
+         show.median = FALSE, band.col = synth.obs.col,
```

```
+         y.col = adjustcolor("grey30", alpha.f = 0.62))
R> polygon(c(1972, 1973, 1973, 1972),
+            c(tail(syn.obs$summary$q025, 1), syn.fore$summary$q025[1],
+                syn.fore$summary$q975[1], tail(syn.obs$summary$q975, 1)),
+            col = synth.fore.col, border = NA)
R> plot(syn.fore, time = 1973:1980, add = TRUE,
+         show.median = FALSE, band.col = synth.fore.col)
R> abline(v = 1972, lty = 3)
R> legend("bottomleft",
+           legend = c("Observed-period synthesis (95%)",
+                      "Forecast synthesis (95%)"),
+           fill = c(synth.obs.col, synth.fore.col), border = NA,
+           bty = "n", cex = 0.66)
```

4.2. Sunspots

For our next example, we use the yearly Sunspot time series from the package **datasets**. The data are yearly counts for sunspots from 1700 to 1988, shown in Figure 3. In this example we show how to use the **dlm** package to create the state-space model, combine blocks of a state-space model, apply the LDVB approximation through `exdqlmLDVB()`, perform visual model diagnostics to compare the exDQLM with the DQLM, and use those diagnostics for discount-factor selection.

Upper-tail sunspot activity is a natural setting for illustrating dynamic quantile modeling of cyclic count data. We therefore model the 0.85 quantile in this example. Again, we begin by building the state-space structure used to estimate the quantile. Here we choose to model the quantile with a first order polynomial trend component as well as a Fourier form seasonal component that incorporates a fundamental period every 11 observations (years), as well as some of its corresponding harmonics. This results in a dynamic model with a total of 9 state parameters, 1 for the first order polynomial component and 8 for the seasonal component. The **exdqlm** functions are compatible with time-invariant `dlm` objects from the **dlm** package. For example, we create the first order trend component with the function `dlmModPoly()` from the **dlm** package, then use `as.exdqlm()` to turn the output into an `exdqlm` object.

```
R> dlm.trend.comp = dlm::dlmModPoly(1, m0 = 50, C0 = 2500)
R> trend.comp = as.exdqlm(dlm.trend.comp)
```

The initial level prior is centered at 50 sunspots, a rounded count-scale value chosen before fitting, with prior standard deviation 50 to keep the level weakly informative on the observed count scale. Next, we construct the seasonal component with our `seasMod()` function. Sunspot cycles are commonly modeled with an approximately 11-year period; we include the first four Fourier harmonics in the component.

```
R> seas.comp = seasMod(p = 11, h = 1:4, C0 = 10 * diag(8))
```

To combine the two components into a single state-space structure, we add the two `exdqlm` objects.

```
R> model = trend.comp + seas.comp
```

The resulting evolution matrix of the combined models is printed for illustration.

```
R> model$GG
```

```
     [,1]    [,2]   [,3]    [,4]   [,5]    [,6]    [,7]    [,8]    [,9]
 [1,]    1  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [2,]    0  0.8413 0.5406  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [3,]    0 -0.5406 0.8413  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [4,]    0  0.0000 0.0000  0.4154 0.9096  0.0000  0.0000  0.0000  0.0000
 [5,]    0  0.0000 0.0000 -0.9096 0.4154  0.0000  0.0000  0.0000  0.0000
 [6,]    0  0.0000 0.0000  0.0000 0.0000 -0.1423  0.9898  0.0000  0.0000
 [7,]    0  0.0000 0.0000  0.0000 0.0000 -0.9898 -0.1423  0.0000  0.0000
 [8,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.6549  0.7557
 [9,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.7557 -0.6549
```

We will apply discount factors by component, i.e. a discount factor of 0.9 for the trend component of dimension 1 and a discount factor of 0.85 for the seasonal component of dimension 8. This discounting strategy can be applied with arguments `df = c(0.9, 0.85)` and `dim.df = c(1, 8)`.

We call the function `exdqlmLDVB()` to obtain the LDVB approximations. We use the routine with argument `dqlm.ind = TRUE` to estimate the DQLM, as well as with the default settings to estimate the exDQLM. In this example we set `n.samp = 3000` and the output is suppressed using input `verbose = FALSE`.

```
R> M1 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9, 0.85), dim.df = c(1, 8),
+                  dqlm.ind = TRUE, fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
R> M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9, 0.85), dim.df = c(1, 8),
+                  fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
```

The fitted objects can be summarized with the `summary()` generic, which reports the model class, sample size, state dimension, discount factors, convergence iterations, and run time. For this dataset of length 289, the DQLM LDVB fit completes more quickly, while the corresponding exDQLM LDVB fit is more computationally demanding because the skewness parameter is estimated rather than fixed. The timing comparison in Table 7 reports the benchmark values used in the manuscript under the recorded backend profile. These runtimes are fit elapsed times stored in the returned fit objects; they are intended for within-profile comparison and should not be interpreted as machine-independent timing constants.

We can visualize the differences between the two resulting dynamic quantiles using the function `exdqlmPlot()`, shown in the lower-left panel of Figure 3. For clarity we limit that panel to years 1780 to 1830.

```
R> plot(sunspot.year, xlim = c(1780, 1830), col = "dark grey",
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(M1, add = TRUE, col = "red")
R> exdqlmPlot(M2, add = TRUE, col = "blue")
R> legend("topleft", lty = 1, bty = "n", col = c("red", "blue"),
+        legend = c("DQLM", "exDQLM"))
R> title("LDVB fit for p0 = 0.85")
```

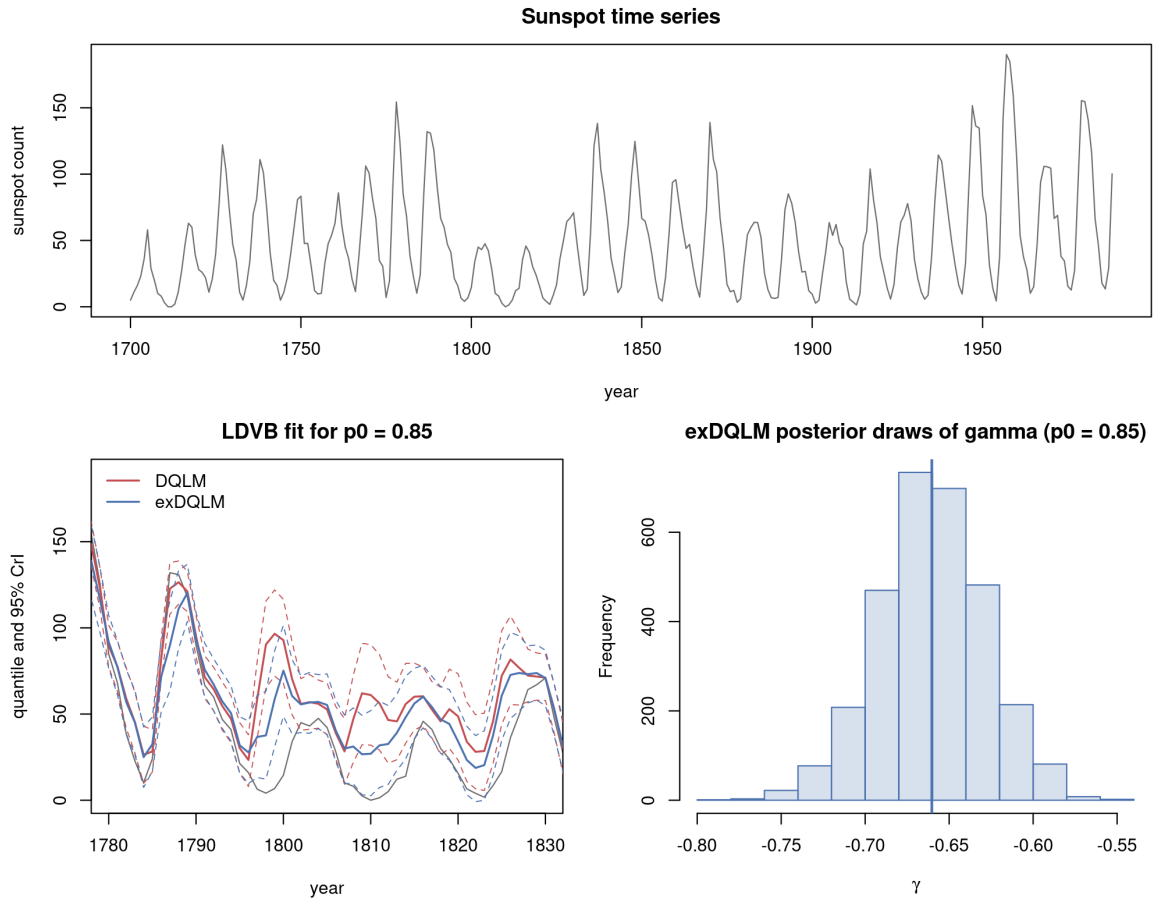

Figure 3: Example 2: Sunspots. Top: The sunspot time series from 1700 to 1988. Bottom left: MAP estimates and 95% CrIs of the estimated quantiles from the DQLM (red) and exDQLM (blue), plotted with the data (grey) from 1780 to 1830. Bottom right: Histogram of samples from the approximated posterior distribution of γ under the exDQLM.

To examine whether the added flexibility of the exDQLM is supported by the fitted approximation, we can also visualize the approximate posterior distribution of the skewness parameter γ by plotting the samples from the corresponding variational distribution, also seen in Figure 3. The approximate posterior mass lies away from zero, supporting the additional skewness flexibility for this upper-quantile fit.

```
R> hist(M2$samp.gamma, xlab = expression(gamma), main = "",
+        col = adjustcolor("#4C72B0", alpha.f = 0.22),
+        border = "#4C72B0")
R> abline(v = median(M2$samp.gamma), col = "#4C72B0", lwd = 2)
R> title("exDQLM posterior draws of gamma (p0 = 0.85)")
```

To further examine the two models, we use the function `exdqlmDiagnostics()` to plot the model diagnostics discussed in Section 3. The results are seen in Figure 4. These diagnostic plots are based on the MAP one-step-ahead standardized forecast errors produced by the filtering recursion, together with the corresponding probability integral transform (PIT)

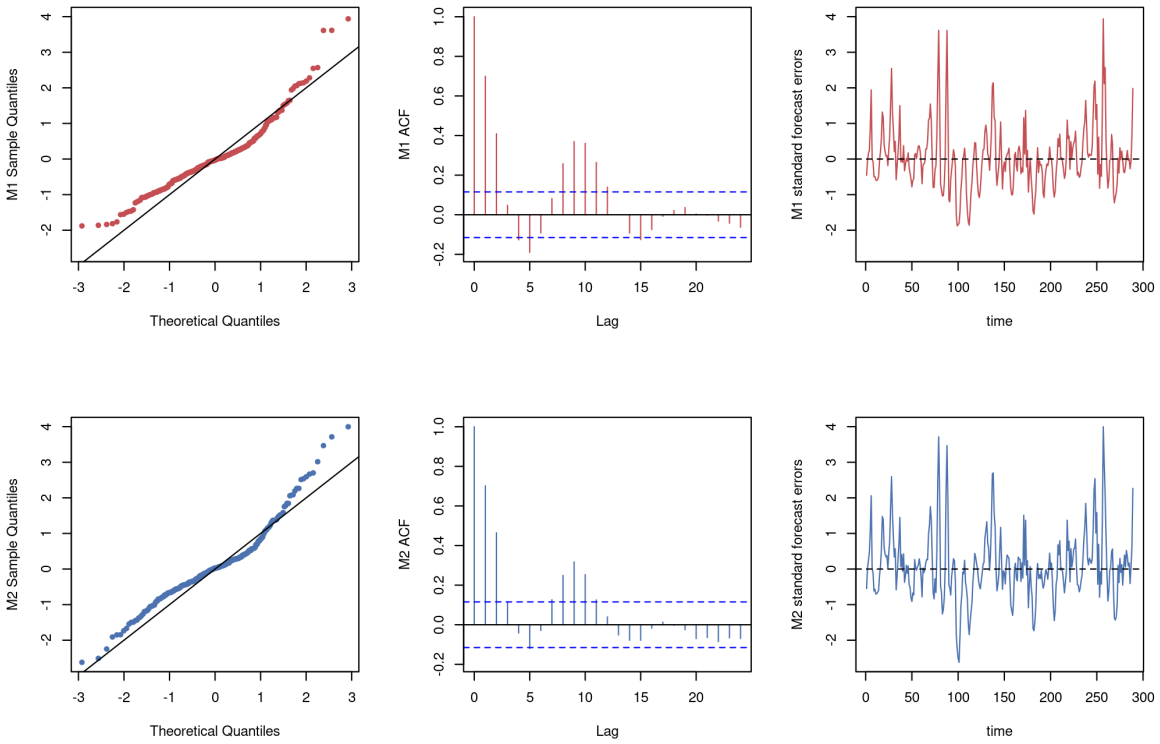

Figure 4: Example 2: Sunspots. The QQ plots (left column), ACF plots of the PIT sequence (center column), and MAP one-step-ahead standardized forecast errors (right column) from the DQLM (red) and exDQLM (blue).

sequence. Thus the figure summarizes the fitted diagnostic sequence rather than posterior bands for residuals.

```
R> par(mfrow = c(2, 3))
R> diagM1M2 = exdqlmDiagnostics(M1, M2, cols = c("red", "blue"))
```

We can also apply the `print()` generic to the resulting `exdqlmDiagnostic` object. Smaller values indicate better fit for the KL, CRPS, and PPLC criteria; for these LDVB fits, the reported diagnostics favor the exDQLM, while the DQLM is faster.

```
R> print(diagM1M2)
```

```
   Diagnostic        M1        M2
           KL    0.1329    0.1197
 KL (flipped)    0.0625    0.0528
         CRPS    8.5349    6.2685
         pplc 3855.9702 2328.0091
 run-time (s)   22.7090  104.0150
```

The function `exdqlmDiagnostics()` can also be used for model selection. Say we want to check whether the discount factor we used for the seasonal component is reasonable under

a predictive scoring rule. We can create a list of possible discount factors, quickly run the LDVB approximation for each possible model, and compare the resulting CRPS and KL values. In practice we recommend selecting the discount factors with the CRPS and using the KL divergence as a complementary calibration check. The selected discount factors are the row with the smallest CRPS.

```
R> possible.dfs = cbind(0.9, seq(0.85, 1, 0.05))
R> possible.dfs

     [,1] [,2]
[1,]  0.9 0.85
[2,]  0.9 0.90
[3,]  0.9 0.95
[4,]  0.9 1.00

R> metrics <- matrix(NA_real_, nrow(possible.dfs), 2,
+                    dimnames = list(NULL, c("CRPS", "KL")))
R> for (i in 1:nrow(possible.dfs)) {
+   temp.M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = possible.dfs[i, ], dim.df = c(1, 8),
+                  sig.init = 2, fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
+   temp.check = exdqlmDiagnostics(temp.M2, plot = FALSE)
+   metrics[i, ] = c(temp.check$m1.CRPS, temp.check$m1.KL)
+ }
R> possible.dfs[which.min(metrics[, "CRPS"]), ]

[1] 0.90 0.85
```

On this coarse grid, the CRPS and KL criteria agree and both select the seasonal discount factor 0.85, so the choice used above is retained.

To compare the fast variational approximation with posterior simulation, we fit matched DQLM and exDQLM specifications with `exdqlmMCMC()`. The MCMC fits use the same state-space model, quantile level, and discount factors as the LDVB fits, with 2000 burn-in iterations and 3000 retained draws.

```
R> M1mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model,
+                 df = c(0.9, 0.85), dim.df = c(1, 8),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                 dqlm.ind = TRUE, fix.sigma = FALSE)
R> M2mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model,
+                 df = c(0.9, 0.85), dim.df = c(1, 8),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                 fix.sigma = FALSE)
```

Table 7 gives a compact runtime-and-diagnostic comparison. Within each model class, LDVB is substantially faster. For both DQLM and exDQLM, LDVB gives the smaller KL value, while MCMC gives smaller CRPS and PPLC values. Comparing model classes within each inferential method, the exDQLM improves the LDVB diagnostics relative to the DQLM, and under MCMC it improves the predictive-score criteria CRPS and PPLC. These results reinforce the importance of reporting runtime together with predictive criteria rather than runtime alone.

model	method	runtime (s)	KL	CRPS	PPLC
DQLM	LDVB	**22.71**	**0.133**	8.535	3856.0
	MCMC	267.88	0.152	**7.582**	**2990.8**
exDQLM	LDVB	**104.02**	**0.120**	6.268	2328.0
	MCMC	325.53	0.164	**5.095**	**2186.3**

Table 7: Example 2: representative dynamic benchmark for the DQLM and exDQLM fits at $p_0 = 0.85$. Each row reports fit runtime together with the deterministic KL normality diagnostic, the mean CRPS from posterior predictive empirical quantiles, and the posterior predictive loss criterion (PPLC). The LDVB rows use `n.samp = 3000`. The MCMC rows use 2000 burn-in iterations and 3000 retained draws. Runtimes are fit elapsed times stored in the returned fit objects and exclude diagnostic calculations, plotting, table construction, and manuscript rendering. They were measured under benchmark profile B on an Intel Xeon E5-4650 CPU with R 4.6.0, **exdqlm** version 1.0.0 at package commit `1c09803`, the C++ MCMC backend in `"fast"` mode, and `exdqlm.cpp_threads = 1L`; the full provenance is recorded in `analysis/manuscript/outputs/tables/benchmark_environment.csv`. Because the columns are the comparison metrics, bold entries indicate the smaller value within each model block; displayed ties are left unbolded.

4.3. Big Tree water flow

For the third example, we use observed average monthly water flow (cubic feet per second) at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA (U.S. Geological Survey 2016). The **exdqlm** package includes these measurements as the monthly time series `BTflow`, which currently extends through March 2026. Here we use the complete overlap between `BTflow` and the package data frame `climateIndices`, giving 432 monthly observations from January 1987 through December 2022. The final 18 months, July 2021 through December 2022, are not used for fitting and are shown later in a conditional forecast display. This example illustrates the transfer-function workflow: building a baseline dynamic quantile model, adding external covariates through the transfer-function state augmentation, examining fitted components, and forecasting conditionally on future covariate values.

We focus on the lower-tail quantile $p_0 = 0.15$, which represents lower monthly-flow behavior on the log scale. As transfer-function inputs, we use two monthly climate indices from `climateIndices`: the Northern Oscillation Index (NOI) and the Atlantic Multidecadal Oscillation (AMO) index. These indices come from the NOAA Physical Sciences Laboratory climate-index collection (NOAA Physical Sciences Laboratory 2026) and provide a compact setting for illustrating external-input effects. The covariates are standardized using the training window only. In the forecast display, NOI and AMO values over the held-out months are treated as known future inputs; the display is therefore conditional on those covariates rather than an operational forecast of the climate indices. The complete overlap used here has 432 monthly observations. The first 414 observations, January 1987 through June 2021, form the training period; the final 18 observations, July 2021 through December 2022, form the forecast holdout. The following chunk shows the essential data alignment. The replication script adds date and completeness checks, uses the same training-window scaling, and writes the aligned modeling data to `analysis/manuscript/outputs/tables/ex3_model_dataset.csv`.

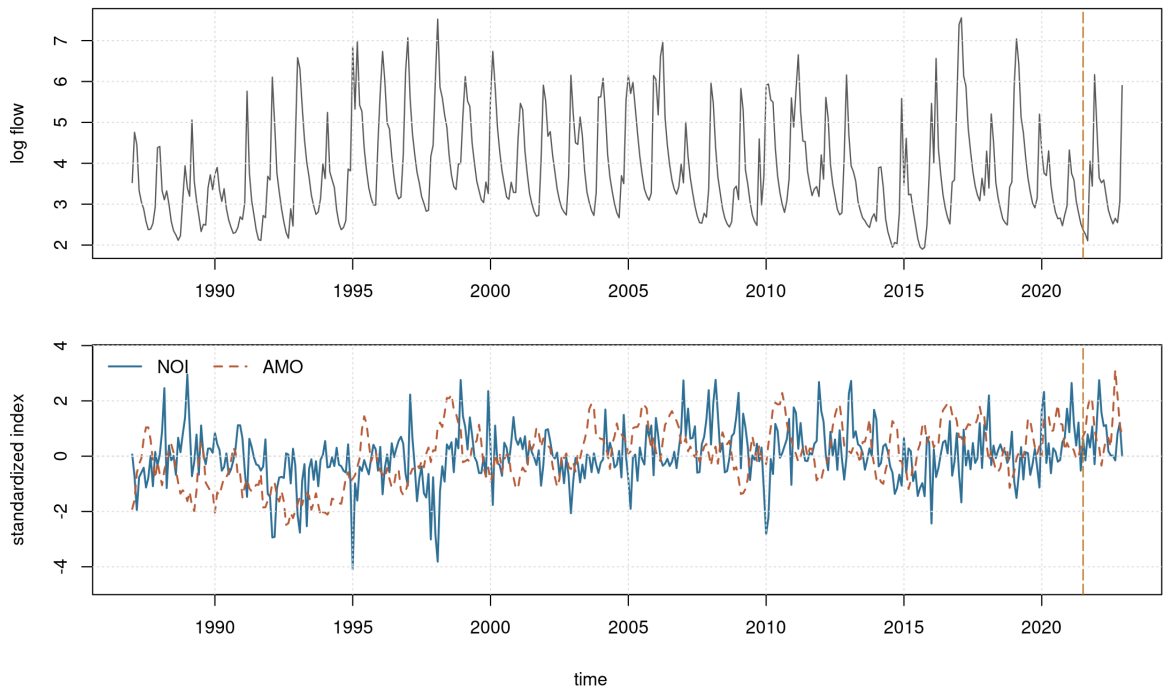

Figure 5: Example 3: Big Tree water flow. Top: observed average monthly water flow at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA, plotted on the log scale. Bottom: standardized monthly values of the Northern Oscillation Index (NOI) and Atlantic Multidecadal Oscillation (AMO) index. The displayed modeling window spans January 1987 through December 2022, and the vertical dashed line marks the beginning of the held-out forecast window in July 2021.

```
R> data("BTflow", package = "exdqlm")
R> data("climateIndices", package = "exdqlm")
R> ex3.dates = seq(as.Date("1987-01-01"), by = "month",
+                  length.out = length(BTflow))
R> flow.df = data.frame(date = ex3.dates, flow = as.numeric(BTflow))
R> ex3.df = merge(flow.df, climateIndices[, c("date", "noi", "amo")],
+                 by = "date")
R> ex3.df = ex3.df[1:432, ]
R> y.fit = ts(log(ex3.df$flow), start = c(1987, 1), frequency = 12)
R> y.train = window(y.fit, end = c(2021, 6))
R> y.holdout = window(y.fit, start = c(2021, 7))
R> X.raw = as.matrix(ex3.df[, c("noi", "amo")])
R> X.train = scale(X.raw[1:414, ])
R> X.holdout = scale(X.raw[415:432, ],
+                    center = attr(X.train, "scaled:center"),
+                    scale = attr(X.train, "scaled:scale"))
```

The baseline state-space model contains a first-order polynomial trend and a Fourier-form seasonal component with period 12. The seasonal block includes $h = 1$, $h = 2$, and $h \approx 0.147$, corresponding to annual, semiannual, and approximately 7-year cycles on the monthly scale.

```
R> trend.comp = polytrendMod(1, m0 = log(50), C0 = 1)
R> seas.comp = seasMod(p = 12, h = c(1, 2, 0.1469118636),
```

```
+                     C0 = diag(1, 6))
R> model = trend.comp + seas.comp
```

The initial log-flow level prior is centered at log(50), corresponding to a fixed 50 cubic feet per second reference value, with prior standard deviation 1 on the log scale. This gives a broad domain-scale prior without using the training response to center the state prior. The standardized climate coefficients are assigned zero-centered normal priors with variance 1. On the log-flow scale this is intentionally weakly informative: it allows either climate index to shift the lower-tail quantile materially while still regularizing the example around no climate effect.

We compare three models built from the same trend and seasonal baseline. The first, `M0`, contains no climate covariates. The second, `MREG`, adds NOI and AMO as direct contemporaneous regressors using `regMod()`. The third, `MTF`, adds the same two covariates through the transfer-function structure from Section 2.4. For all three models, the trend and seasonal discount factors are set to 0.99. To keep the climate-effect comparison simple, the direct-regression coefficients in `MREG` and the instantaneous transfer coefficients ψ_t in `MTF` use discount factor 1, so these coefficient states are static over time. The transfer accumulated-effect state ζ_t retains discount factor 0.99.

The transfer rate λ is selected using the training period only. Guided by an initial coarse scan, the replication script fits the transfer model over $\lambda \in \{0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 0.99\}$ and chooses the value with the smallest posterior predictive loss criterion (PPLC) from `exdqlmDiagnostics()`. The held-out months are not used in this selection step. The following code shows the selection pattern used by the replication script.

```
R> lambda.grid = c(0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 0.99)
R> pplc.grid = rep(NA_real_, length(lambda.grid))
R> for (i in seq_along(lambda.grid)) {
+     set.seed(20264001 + i)
+     temp.MTF = exdqlmTransferLDVB(y = y.train, p0 = 0.15, model = model,
+                   df = c(0.99, 0.99), dim.df = c(1, 6),
+                   X = X.train, tf.df = c(0.99, 1),
+                   lam = lambda.grid[i], tf.m0 = rep(0, 3),
+                   tf.C0 = diag(c(0.1, 1, 1), 3),
+                   sig.init = 0.1, gam.init = -0.1,
+                   n.samp = 1000, tol = 0.05, verbose = FALSE)
+     temp.diag = exdqlmDiagnostics(temp.MTF, plot = FALSE)
+     pplc.grid[i] = temp.diag$m1.pplc
+   }
R> lambda.star = lambda.grid[which.min(pplc.grid)]
R> lambda.star

[1] 0.85
```

The final models are fit on January 1987 through June 2021. We use the LDVB implementation for all three models; the corresponding MCMC implementation is available when posterior simulation is desired.

```
R> set.seed(20264101)
R> M0 = exdqlmLDVB(y = y.train, p0 = 0.15, model = model,
+                  df = c(0.99, 0.99), dim.df = c(1, 6),
```

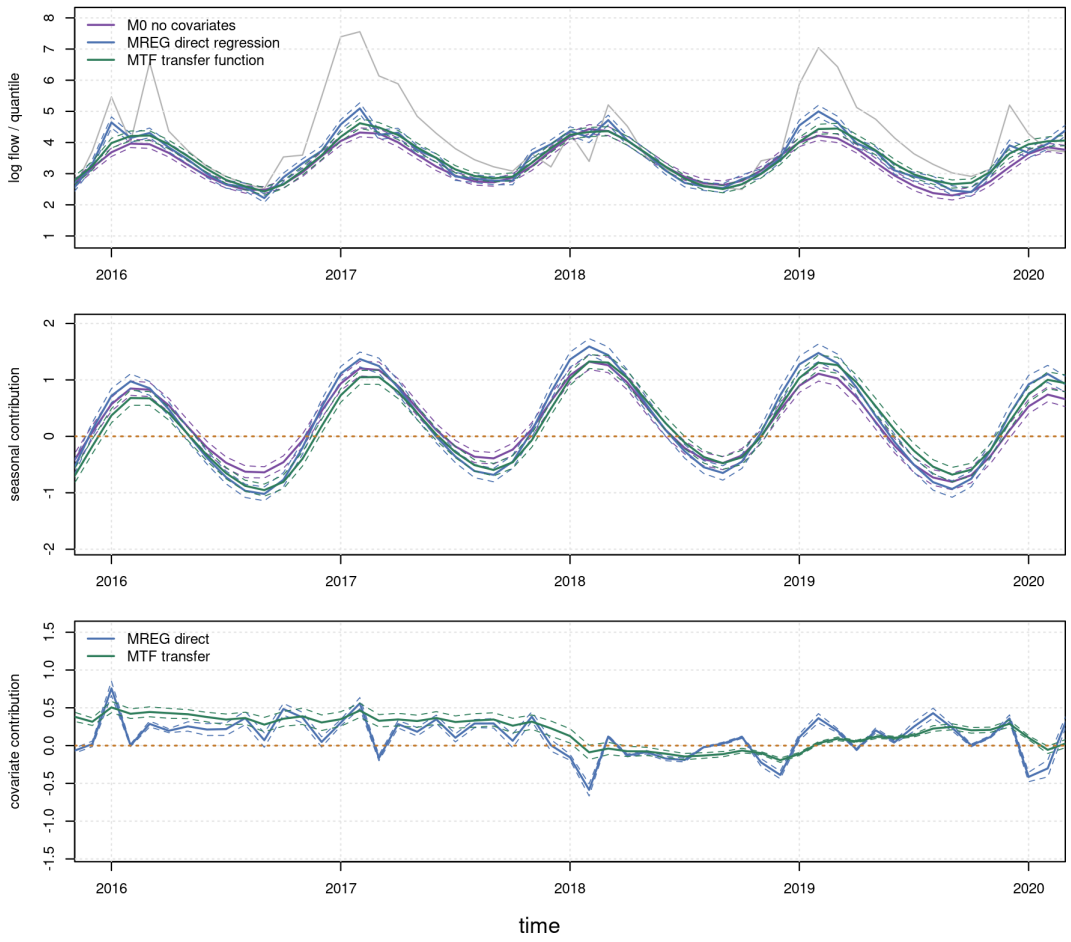

Figure 6: Example 3: Big Tree water flow. Top: MAP estimates and 95% CrIs of the fitted 0.15 quantile from the no-covariate baseline `M0` (purple), direct-regression model `MREG` (blue), and transfer-function model `MTF` (green), plotted with the data (grey) from 2016 to 2020. Middle: MAP estimates and 95% CrIs of the combined seasonal components implied by the annual, semiannual, and low-frequency harmonics. Bottom: MAP estimates and 95% CrIs of the direct-regression contribution in `MREG` and the transfer-function contribution in `MTF`. The horizontal orange dotted lines are at zero for reference.

```
+                sig.init = 0.1, gam.init = -0.1,
+                n.samp = 1000, tol = 0.05, verbose = FALSE)
R> reg.comp = regMod(X.train, m0 = rep(0, 2), C0 = diag(1, 2))
R> set.seed(20264301)
R> MREG = exdqlmLDVB(y = y.train, p0 = 0.15,
+                   model = model + reg.comp,
+                   df = c(0.99, 0.99, 1), dim.df = c(1, 6, 2),
+                   sig.init = 0.1, gam.init = -0.1,
+                   n.samp = 1000, tol = 0.05, verbose = FALSE)
R> set.seed(20264201)
R> MTF = exdqlmTransferLDVB(y = y.train, p0 = 0.15, model = model,
+                           df = c(0.99, 0.99), dim.df = c(1, 6),
+                           X = X.train, tf.df = c(0.99, 1),
```

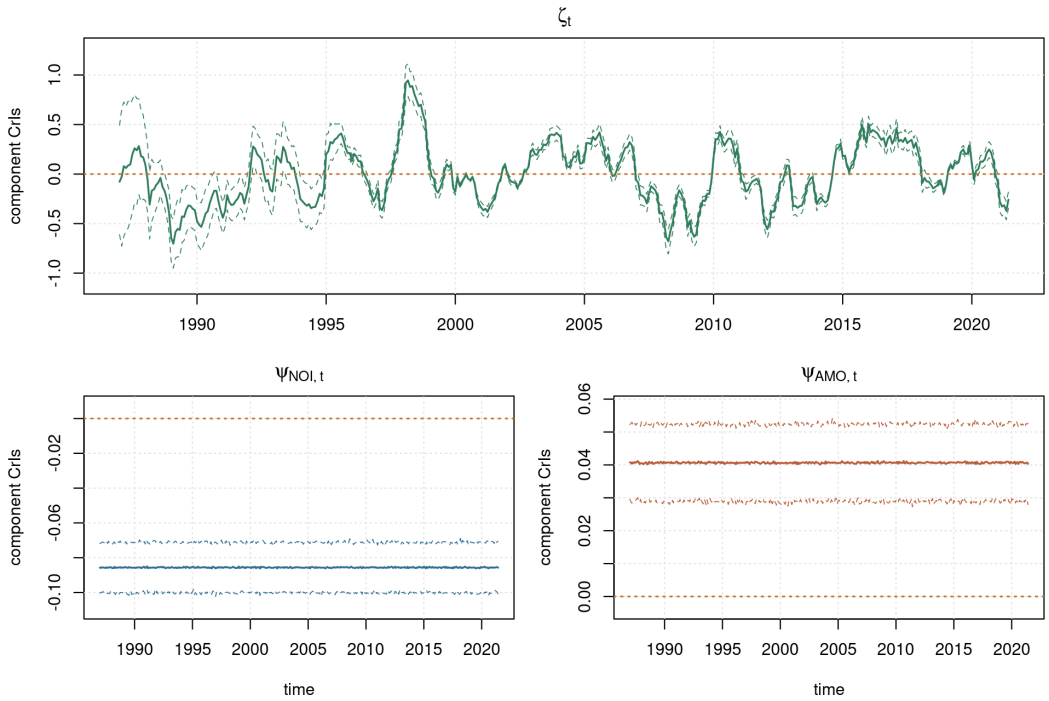

Figure 7: Example 3: Big Tree water flow. MAP estimates and 95% CrIs of the transfer-function states in `MTF`. The top panel shows the accumulated transfer effect ζ_t. The bottom panels show the instantaneous transfer coefficients $\psi_{\mathrm{NOI},t}$ and $\psi_{\mathrm{AMO},t}$, which are static here because their discount factors are set to 1. Horizontal dotted orange lines mark zero.

```
+                         lam = lambda.star, tf.m0 = rep(0, 3),
+                         tf.C0 = diag(c(0.1, 1, 1), 3),
+                         sig.init = 0.1, gam.init = -0.1,
+                         n.samp = 1000, tol = 0.05, verbose = FALSE)
```

The upper panel of Figure 6 compares the fitted 0.15 quantiles from the three training fits over 2016–2020. We first plot the data, then call `exdqlmPlot()` with `add = TRUE` to draw the three model fits on common axes.

```
R> par(mfrow = c(3, 1), mar = c(2.8, 4.4, 1.0, 0.9),
+     oma = c(1.8, 0, 0, 0))
R> plot(y.train, col = "grey", ylim = c(1, 8), xlim = c(2016, 2020),
+         ylab = "log flow / quantile")
R> grid(col = "grey90")
R> exdqlmPlot(M0, add = TRUE)
R> exdqlmPlot(MREG, add = TRUE, col = "steelblue")
R> exdqlmPlot(MTF, add = TRUE, col = "forest green")
R> legend("topleft",
+        legend = c("M0 no covariates", "MREG direct regression",
+                   "MTF transfer function"),
+        col = c("purple", "steelblue", "forest green"),
+        lty = 1, lwd = 1.5, bty = "n")
```

The fitted lower-tail quantile paths are similar because the trend and seasonal components explain much of the broad annual structure. The component panels of Figure 6 clarify the

model differences: the middle panel shows that the seasonal contribution is similar across models, while the lower panel separates the contemporaneous climate-index contribution in `MREG` from the accumulated transfer contribution in `MTF`. The package function `compPlot()` is used to display these selected model components. We begin with the combined seasonal contribution. Because the seasonal block contains three harmonics, this contribution is formed from the second through seventh elements of the state vector. The middle panel of Figure 6 shows the resulting seasonal estimates for the three models over the same 2016–2020 window.

```
R> plot(NA, ylim = c(-2, 2), xlim = c(2016, 2020),
+      ylab = "seasonal components")
R> grid(col = "grey90")
R> compPlot(M0, index = 2:7, add = TRUE)
R> compPlot(MREG, index = 2:7, add = TRUE, col = "steelblue")
R> compPlot(MTF, index = 2:7, add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
```

The lower panel of Figure 6 displays the climate-index contributions induced by the standardized NOI and AMO covariates. For `MREG`, indices 8 and 9 are the direct-regression coefficient states. For `MTF`, index 8 is the accumulated transfer effect ζ_t. This separates contemporaneous and lagged climate-index components from the common trend and seasonal baseline, even when the fitted quantiles themselves are visually close.

```
R> plot(NA, ylim = c(-1.5, 1.5), xlim = c(2016, 2020),
+      ylab = "covariate contribution")
R> grid(col = "grey90")
R> compPlot(MREG, index = 8:9, add = TRUE, col = "steelblue")
R> compPlot(MTF, index = 8, add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
R> legend("topleft", legend = c("MREG direct", "MTF transfer"),
+        col = c("steelblue", "forest green"), lty = 1, lwd = 1.5,
+        bty = "n")
```

Figure 7 then displays the augmented transfer states directly. As described in Section 2.4, ζ_t controls the accumulated transfer effect and the ψ_t states determine how the contemporaneous covariates enter that transfer term. In this example the ψ_t discount factors are set to 1, so the bottom panels summarize static NOI and AMO coefficients within the transfer-function model. Setting `just.theta = TRUE` in `compPlot()` plots a single element of the augmented state vector $\tilde{\boldsymbol{\theta}}_t$. In this fitted model, indices 8, 9, and 10 correspond to ζ_t, $\psi_{\mathrm{NOI},t}$, and $\psi_{\mathrm{AMO},t}$, respectively.

```
R> layout(matrix(c(1, 1, 2, 3), nrow = 2, byrow = TRUE))
R> compPlot(MTF, index = 8, col = "forest green",
+             add = FALSE, just.theta = TRUE)
R> grid(col = "grey90")
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
R> title(expression(zeta[t]))
R> par(mar = c(3.8, 4.2, 2.1, 0.8))
R> plot(y.train, type = "n", ylim = c(-0.11, 0.01),
+      xlab = "time", ylab = "component CrIs")
R> compPlot(MTF, index = 9, col = "steelblue",
+             add = TRUE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 1.4)
```

```
R> title(expression(psi[list(NOI, t)]))
R> plot(y.train, type = "n", ylim = c(-0.005, 0.06),
+         xlab = "time", ylab = "component CrIs")
R> compPlot(MTF, index = 10, col = "darkorange",
+             add = TRUE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(psi[list(AMO, t)]))
```

The fitted object also reports `median.kt`, the median number of time steps until the transfer effect on the 0.15 quantile is less than or equal to 1e−3, as discussed in Section 2.4. Here `median.kt` is approximately 25.15, indicating a persistent transfer effect under the selected λ.

```
R> MTF$median.kt

[1] 25.15187
```

We next produce 18-month-ahead forecasts over the held-out window. The future state-space matrices are passed through the `fFF` and `fGG` arguments of `exdqlmForecast()`. The code below shows the relevant matrix construction and forecast calls; the replication script wraps these steps in dimension checks and output-writing code. For `M0`, the future state-space matrices repeat the trend and seasonal model.

```
R> k.fore = length(y.holdout)
R> F0.future = model$FF
R> G0.future = model$GG
R> fc.M0 = exdqlmForecast(start.t = length(y.train), k = k.fore, m1 = M0,
+                         fFF = F0.future, fGG = G0.future,
+                         return.draws = TRUE, n.samp = 1000,
+                         seed = 20265101, plot = FALSE)
```

For `MREG`, the future observation matrix includes the held-out NOI and AMO values as direct regressors.

```
R> n.x = ncol(X.holdout)
R> reg.future = regMod(X.holdout, m0 = rep(0, n.x), C0 = diag(1, n.x))
R> model.reg.future = model + reg.future
R> FREG.future = model.reg.future$FF
R> GREG.future = model.reg.future$GG
R> fc.MREG = exdqlmForecast(start.t = length(y.train), k = k.fore, m1 = MREG,
+                           fFF = FREG.future, fGG = GREG.future,
+                           return.draws = TRUE, n.samp = 1000,
+                           seed = 20265301, plot = FALSE)
```

For `MTF`, the future matrices include the held-out NOI and AMO values through the transfer-function augmentation.

```
R> n.state = length(model$m0)
R> FTF.future = matrix(0, nrow = n.state + n.x + 1, ncol = k.fore)
R> FTF.future[seq_len(n.state), ] = F0.future
R> FTF.future[n.state + 1, ] = 1
R> GTF.future = array(0, c(n.state + n.x + 1,
```

```
+                              n.state + n.x + 1, k.fore))
R> GTF.future[seq_len(n.state), seq_len(n.state), ] = G0.future
R> zeta.ind = n.state + 1
R> psi.ind = n.state + 1 + seq_len(n.x)
R> GTF.future[zeta.ind, zeta.ind, ] = lambda.star
R> for (j in seq_len(n.x)) {
+    GTF.future[zeta.ind, psi.ind[j], ] = X.holdout[, j]
+    GTF.future[psi.ind[j], psi.ind[j], ] = 1
+    }
R> fc.MTF = exdqlmForecast(start.t = length(y.train), k = k.fore, m1 = MTF,
+                              fFF = FTF.future, fGG = GTF.future,
+                              return.draws = TRUE, n.samp = 1000,
+                              seed = 20265201, plot = FALSE)
```

Finally, Figure 8 overlays the filtered estimates before the forecast origin and the conditional forecasts after it.

```
R> plot(y.fit, col = "grey70", xlim = c(2020, 2023), ylim = c(1,8),
        ylab = "log flow / forecast quantile", xlab = "time")
R> grid(col = "grey90")
R> plot(fc.M0, add = TRUE, cols = c("purple", "plum"))
R> plot(fc.MREG, add = TRUE, cols = c("steelblue", "lightblue"))
R> plot(fc.MTF, add = TRUE, cols = c("forest green", "darkseagreen"))
R> t.all = as.numeric(time(y.fit))
R> hold.idx = 415:432
R> lines(t.all[hold.idx], y.fit[hold.idx],
+           col = "darkorange", lwd = 1.4)
R> points(t.all[hold.idx], y.fit[hold.idx],
+            col = "darkorange", pch = 1, cex = 0.8)
R> abline(v = t.all[hold.idx[1]], col = "orange", lty = 5, lwd = 1.2)
R> legend("topleft",
+            legend = c("M0 no covariates", "MREG direct regression",
+                       "MTF transfer", "held-out observations"),
+            col = c("purple", "steelblue", "forest green", "darkorange"),
+            lty = 1, pch = c(NA, NA, NA, 1), bty = "n")
```

The numerical comparison in Table 8 uses `exdqlmDiagnostics()` so that all reported quantities are exported package diagnostics, as defined in Section 3. These diagnostics are computed from the final-training fitted objects.

```
R> diag.M0 = exdqlmDiagnostics(M0, plot = FALSE)
R> diag.MREG = exdqlmDiagnostics(MREG, plot = FALSE)
R> diag.MTF = exdqlmDiagnostics(MTF, plot = FALSE)
R> tab.ex3 = data.frame(
+     model = c("M0", "MREG", "MTF"),
+     KL = c(diag.M0$m1.KL, diag.MREG$m1.KL, diag.MTF$m1.KL),
+     CRPS = c(diag.M0$m1.CRPS, diag.MREG$m1.CRPS, diag.MTF$m1.CRPS),
+     PPLC = c(diag.M0$m1.pplc, diag.MREG$m1.pplc, diag.MTF$m1.pplc))
```

Alternatively, Table 9 reports held-out forecast scores over the final 18 months. These diagnostics are computed from the forecast objects using `exdqlmForecastDiagnostics()`.

```
R> fc.diag.M0 = exdqlmForecastDiagnostics(fc.M0, y = y.holdout)
R> fc.diag.MREG = exdqlmForecastDiagnostics(fc.MREG, y = y.holdout)
```

Model	KL	CRPS	PPLC
`M0` no covariates	0.168	0.471	**99.459**
`MREG` direct regression	**0.112**	**0.347**	151.171
`MTF` transfer function	0.155	0.366	125.430

Table 8: Example 3: Big Tree water flow. Package diagnostics from `exdqlmDiagnostics()` for the no-covariate baseline `M0`, direct-regression model `MREG`, and transfer-function model `MTF`, computed on the final-training fitted objects. KL denotes the deterministic Kullback–Leibler normality diagnostic for the MAP standardized forecast errors, CRPS denotes the continuous ranked probability score, and PPLC denotes the posterior predictive loss criterion. Lower values are preferred; bold entries mark the smallest value for each diagnostic.

```
R> fc.diag.MTF = exdqlmForecastDiagnostics(fc.MTF, y = y.holdout)
R> tab.ex3.fc = data.frame(
+     model = c("M0", "MREG", "MTF"),
+     check.loss = c(fc.diag.M0$m1.check_loss,
+                    fc.diag.MREG$m1.check_loss,
+                    fc.diag.MTF$m1.check_loss),
+     CRPS = c(fc.diag.M0$m1.CRPS,
+              fc.diag.MREG$m1.CRPS,
+              fc.diag.MTF$m1.CRPS))
```

Table 8 gives a mixed in-sample diagnostic comparison. The direct-regression model has the smallest KL and CRPS values, while the no-covariate baseline has the smallest PPLC. The transfer-function model falls between them for PPLC and has diagnostics closer to `MREG` than to `M0` for CRPS. The example instead shows how **exdqlm** supports three related dynamic quantile workflows: no external inputs, direct regression through `regMod()`, and lagged external-input effects through transfer functions.

Model	Check loss	CRPS
`M0` no covariates	0.119	0.498
`MREG` direct regression	**0.114**	**0.401**
`MTF` transfer function	0.155	0.550

Table 9: Example 3: Big Tree water flow. Held-out forecast metrics over July 2021 through December 2022, computed with `exdqlmForecastDiagnostics()` from `exdqlmForecast(..., return.draws = TRUE)` objects. Check loss is computed at $p_0 = 0.15$, and CRPS is computed from posterior predictive forecast draws using the integrated-quantile-score approximation. Lower values are preferred; bold entries mark the smallest value in each column.

4.4. Static exAL regression on a simulated sparse Gaussian benchmark

To illustrate the static routines under sparse shrinkage, we consider a correlated Gaussian regression benchmark for which the target conditional quantile is known exactly. Let

$$\mathbf{x}_i \sim \mathcal{N}_8(\mathbf{0}, \Sigma_x), \qquad (\Sigma_x)_{jk} = 0.5^{|j-k|},$$

with sparse signal

$$\boldsymbol{\beta}^\star = (3,\, 1.5,\, 0,\, 0,\, 2,\, 0,\, 0,\, 0)^\top.$$

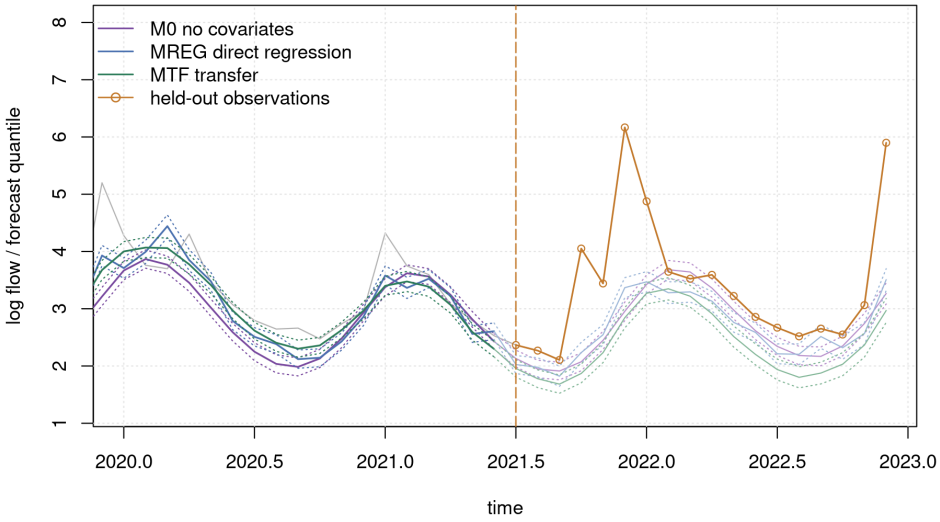

Figure 8: Example 3: Big Tree water flow. Held-out 18-month conditional forecast display from July 2021 through December 2022. The vertical dot-dashed line marks the forecast origin after the final training month, June 2021. Solid lines indicate posterior mean forecast quantiles and dotted lines indicate pointwise 95% CrIs for `M0`, `MREG`, and `MTF`. Held-out observations are shown as orange circles.

We use the following benchmark data-generating mechanism. For each target quantile level p_0, we generate

$$Y_i^{(p_0)} = \mathbf{x}_i^\top \boldsymbol{\beta}^\star + \sigma_\varepsilon\{Z_i - \Phi^{-1}(p_0)\}, \qquad Z_i \sim \mathcal{N}(0,1), \qquad \sigma_\varepsilon = 1.5.$$

Then

$$Q_Y(p_0 \mid \mathbf{x}_i) = \mathbf{x}_i^\top \boldsymbol{\beta}^\star,$$

so the true p_0-quantile is the same sparse linear signal at each fitted quantile level. This is a quantile-specific calibration benchmark: each target level is generated so that the known target quantile is the same sparse linear signal, rather than a single-response multi-quantile analysis. We use $n = 160$ training observations and an independent holdout sample of size 800, and fit separate static exAL models at $p_0 \in \{0.05, 0.25, 0.50\}$. Both fits use the Nishimura–Suchard regularized horseshoe prior (Nishimura and Suchard 2023), implemented in the package through `beta_prior = "rhs_ns"`, with $\tau_0 = 0.15$, fixed slab $\zeta^2 = 9$, and an unshrunk intercept. We fit an LDVB approximation with `exalStaticLDVB()` using `n.samp = 3000`, and an MCMC fit with `exalStaticMCMC()` using `n.burn = 2000` and `n.mcmc = 3000`; the MCMC routine uses its default `slice` kernel and is warm-started from the LDVB fit. The lower-tail LDVB fit at $p_0 = 0.05$ converges more slowly, so we allow a larger LDVB iteration budget there. Setting `al.ind = TRUE` (or equivalently `dqlm.ind = TRUE`) would recover the AL special case discussed in Section 2, but here we focus on the general static exAL model under sparse `rhs_ns` shrinkage. Gaussian design matrices are generated with `MASS::mvrnorm()` from **MASS** (Venables and Ripley 2002).

```
R> Sigma.x = 0.5 ^ as.matrix(dist(1:8))
R> beta.true = c(3, 1.5, 0, 0, 2, 0, 0, 0)
R> rhs.ctrl = list(tau0 = 0.15, a_zeta = 2, b_zeta = 9,
+                  zeta2_fixed = 9, shrink_intercept = FALSE)
R> set.seed(20260712)
R> sim.y = function(X.raw, p0) {
+     drop(X.raw %*% beta.true) + 1.5 * (rnorm(nrow(X.raw)) - qnorm(p0))
+   }
R> X.raw = MASS::mvrnorm(160, mu = rep(0, 8), Sigma = Sigma.x)
R> X = cbind(1, X.raw)
R> X.hold.raw = MASS::mvrnorm(800, mu = rep(0, 8), Sigma = Sigma.x)
R> X.hold = cbind(1, X.hold.raw)
R> ref.hold = drop(X.hold %*% c(0, beta.true))
R> y.hold = sim.y(X.hold.raw, 0.25)
R> p.grid = c(0.05, 0.25, 0.50)
R> y.train = lapply(p.grid, function(p0) sim.y(X.raw, p0))
R> M.ldvb = lapply(p.grid, function(p0) exalStaticLDVB(
+                   y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
+                   beta_prior = "rhs_ns",
+                   beta_prior_controls = rhs.ctrl,
+                   max_iter = ifelse(p0 == 0.05, 420, 260),
+                   n.samp = 3000,
+                   tol = 1e-4, verbose = FALSE))
R> M.mcmc = lapply(p.grid, function(p0) exalStaticMCMC(
+                   y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
+                   beta_prior = "rhs_ns",
+                   beta_prior_controls = rhs.ctrl,
+                   n.burn = 2000, n.mcmc = 3000,
+                   init.from.vb = TRUE,
+                   verbose = FALSE))
```

For static fits, the helper function `exalStaticDiagnostics()` summarizes fitted quantiles on a common design matrix and, when holdout responses or a reference quantile are supplied, reports predictive evaluation metrics such as check loss and reference-quantile error. It also stores posterior coefficient summaries that can be plotted with the `plot()` method. In this simulated example, the true coefficient vector is known, so Figure 9 overlays it as a visual reference; this truth overlay is optional and is not needed for real data applications. The active-signal RMSE and null-coefficient MAE in Table 10 are simulation-benchmark summaries computed separately from the package diagnostics.

```
R> y.hold = lapply(p.grid, function(p0) sim.y(X.hold.raw, p0))
R> diag.static = Map(function(ldvb, mcmc, y.h) {
+     exalStaticDiagnostics(ldvb, mcmc,
+                           X = X.hold, y = y.h, ref = ref.hold,
+                           plot = FALSE)
+   }, M.ldvb, M.mcmc, y.hold)
R> y.lim = range(beta.true,
+                unlist(lapply(diag.static, function(z) {
+                  c(z$m1.beta.lb[-1], z$m1.beta.ub[-1],
+                    z$m2.beta.lb[-1], z$m2.beta.ub[-1])
+                })))
R> y.lim = y.lim + c(-1, 1) * 0.08 * diff(y.lim)
R> par(mfrow = c(1, 3), mar = c(5.2, 4.0, 2.6, 1.0), xpd = NA)
R> for (i in seq_along(p.grid)) {
+     plot(diag.static[[i]], type = "coefficients",
+          beta.ref = c(0, beta.true),
```

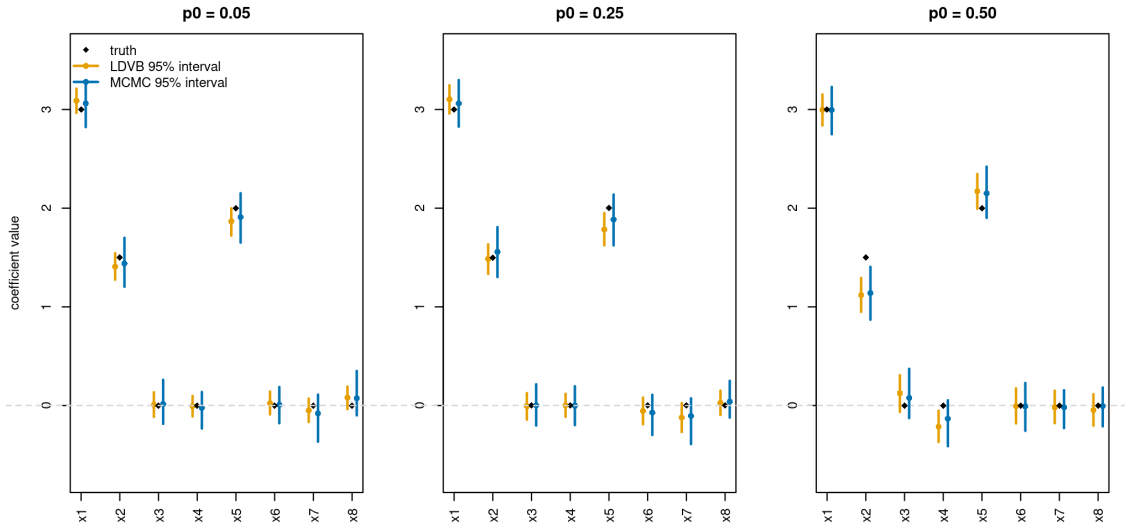

Figure 9: Example 4: sparse static exAL regression under the correlated Gaussian benchmark with the `rhs_ns` prior. Black diamonds denote the true non-intercept coefficients, orange intervals denote LDVB 95% posterior intervals, and blue intervals denote MCMC 95% posterior intervals. Panels correspond to $p_0 = 0.05, 0.25$, and 0.50.

```
+          include.intercept = FALSE,
+          coef.names = c("(Intercept)", paste0("x", seq_along(beta.true))),
+          cols = c("orange", "steelblue"),
+          legend.labels = c("LDVB 95% interval", "MCMC 95% interval"),
+          beta.ref.label = "truth", ylim = y.lim,
+          ylab = if (i == 1) "coefficient value" else "",
+          main = sprintf("p0 = %.2f", p.grid[i]), legend = i == 1)
+   }
```

Figure 9 shows that both methods recover the active coefficients and shrink the null coefficients toward zero across the lower tail, lower-middle quantile, and median. The lower-tail fit at $p_0 = 0.05$ remains the most demanding case, but both methods still recover the sparse signal structure. Across the three panels, the MCMC intervals tend to give slightly cleaner active-signal recovery, while the LDVB intervals provide a fast approximation to the same coefficient-level summaries.

Table 10 reinforces the visual comparison. The MCMC reference fit has the smaller holdout quantile RMSE and active-signal RMSE at all three quantile levels, while LDVB is slightly smaller on null-coefficient MAE at $p_0 = 0.05$ and $p_0 = 0.25$. The LDVB fit is nevertheless substantially faster in all three cases, with runtime reductions of roughly 7-fold at $p_0 = 0.05$, 14-fold at $p_0 = 0.25$, and 65-fold at $p_0 = 0.50$. In this benchmark, the LDVB approximation remains useful as a rapid first-pass screen, but the MCMC fit provides the cleaner holdout quantile accuracy. Accordingly, the static `rhs_ns` specification still supports both roles cleanly: LDVB for fast screening and model exploration, and static MCMC as the reference method when higher-fidelity posterior uncertainty quantification is required.

p_0	method	runtime (s)	active RMSE	null MAE	holdout qRMSE
0.05	LDVB	**27.96**	0.107	**0.034**	0.674
	MCMC	196.52	**0.072**	0.040	**0.592**
0.25	LDVB	**13.89**	0.138	**0.042**	0.393
	MCMC	192.51	**0.082**	0.044	**0.328**
0.50	LDVB	**3.11**	0.241	0.081	0.410
	MCMC	201.30	**0.225**	**0.048**	**0.372**

Table 10: Example 4: runtime and sparse-signal recovery metrics for the static exAL fits under the `rhs_ns` prior. The LDVB rows use `n.samp = 3000`, and the MCMC rows use 2000 burn-in iterations and 3000 retained draws. The active RMSE is computed over the three nonzero coefficients, the null MAE is computed over the five zero coefficients, and the holdout quantile RMSE is computed against the true target quantile on an independent holdout sample of size 800. Because the columns are the comparison metrics, bold entries indicate the smaller value within each quantile block; displayed ties are left unbolded.

5. Conclusion

exdqlm provides an R workflow for Bayesian quantile regression with primary emphasis on latent state-space models for a single response quantile. The package is motivated by a gap between broad quantile-regression tools, which do not provide the dedicated latent quantile-state workflow targeted here, and Gaussian dynamic-modeling packages, which do not target AL or exAL quantile likelihoods. Within this setting, **exdqlm** fits DQLMs and exDQLMs, offers MCMC posterior simulation and LDVB approximate inference, and uses the resulting fitted objects for state and component summaries, forecasting, calibration and predictive diagnostics including deterministic KL, CRPS, and PPLC, and posterior-predictive synthesis across separately fitted quantile levels. The package also includes transfer-function state augmentation and static AL/exAL regression with shrinkage priors.

The examples illustrate how these pieces are used in analysis. Lake Huron uses separate dynamic fits at three quantile levels, multi-step forecasting, and post hoc synthesis of posterior predictive draws. Sunspots compares DQLM and exDQLM fits at an upper quantile, uses MAP one-step-ahead calibration diagnostics together with CRPS and PPLC, and illustrates discount-factor selection by CRPS with KL used as a complementary calibration check. Big Tree water flow compares no-covariate, direct-regression, and transfer-function specifications built from a common trend-seasonal baseline; its fitted and held-out summaries are mixed, so the example is mainly a workflow comparison rather than evidence for a uniformly preferred covariate structure. The static simulation is a quantile-specific sparse exAL benchmark comparing LDVB and MCMC under a known target quantile.

The examples also delineate the current scope of the package. Dynamic models are fitted one quantile level at a time. `quantileSynthesis()` combines predictive draws after separate fits and applies monotonicity adjustments when needed; it is not a joint posterior model over multiple quantile levels. LDVB returns samples from fitted variational factors rather than from the exact posterior target, so MCMC remains the reference calculation when posterior tail summaries, interval widths, or approximation error are central. In transfer-function models, lag propagation is controlled by a fixed or grid-selected λ; the current implementation

does not estimate input-specific lag-decay rates internally. Runtime results are elapsed fitting times under the stated backend profile and should be read alongside predictive and calibration diagnostics.

Several development directions follow from these limitations: multivariate-output dynamic quantile models that represent cross-series dependence through a modeled correlation matrix for related time series; improved approximations for the nonconjugate joint (γ, σ) skewness–scale block; nonlinear dynamic quantile modeling beyond the linear state-space form used here; and joint or regularized fitting across quantile levels. The main article, supplement, and replication materials serve separate roles: the article describes the user-facing workflow, the supplement records posterior targets and algorithmic details, and the repository scripts regenerate the reported figures, tables, example artifacts, and benchmark provenance.

References

Barata R, Prado R, Sansó B (2022). "Fast inference for time-varying quantiles via flexible dynamic models with application to the characterization of atmospheric rivers." *The Annals of Applied Statistics*, **16**(1), 247–271. doi:10.1214/21-AOAS1497.

Barlow RE, Bartholomew DJ, Bremner JM, Brunk HD (1972). *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. John Wiley & Sons, New York.

Beal MJ (2003). *Variational algorithms for approximate Bayesian inference*. Ph.D. thesis, UCL (University College London).

Benoit D, Al-Hamzawi R, Yu K, Van den Poel D (2023). ***bayesQR**: Bayesian Quantile Regression*. R package version 2.4, URL https://CRAN.R-project.org/package=bayesQR.

Bürkner PC (2026). ***brms**: Special Family Functions for **brms** Models*. Package documentation, accessed 2026-04-19, URL https://paulbuerkner.com/brms/reference/brmsfamily.html.

Carter CK, Kohn R (1994). "On Gibbs sampling for state space models." *Biometrika*, **81**(3), 541–553.

Chernozhukov V, Fernández-Val I, Galichon A (2010). "Quantile and Probability Curves Without Crossing." *Econometrica*, **78**(3), 1093–1125. doi:10.3982/ECTA7880.

Fan K, Wu C, Ren J, Li X, Zhou F (2026). ***pqrBayes**: Bayesian Penalized Quantile Regression*. R package version 1.2.1, URL https://CRAN.R-project.org/package=pqrBayes.

Fasiolo M, Griffiths B (2025). ***qgam**: Smooth Additive Quantile Regression Models*. R package version 2.0.0, URL https://CRAN.R-project.org/package=qgam.

Fogarty B (2020). "**QuantReg.jl**: Quickstart Guide." Documentation page, accessed 2026-04-19, URL https://fogarty-ben.github.io/QuantReg.jl/v0.1/quickstart/.

Frühwirth-Schnatter S (1994). "Data augmentation and dynamic linear models." *Journal of time series analysis*, **15**(2), 183–202.

Frumento P (2025). ***qrcm**: Quantile Regression Coefficients Modeling*. R package version 3.2, URL https://CRAN.R-project.org/package=qrcm.

Galarza CE, Benites L, Bourguignon M, Lachos VH (2024). ***lqr**: Robust Linear Quantile Regression*. R package version 5.2, URL https://CRAN.R-project.org/package=lqr.

Gelfand AE, Ghosh SK (1998). "Model choice: a minimum posterior predictive loss approach." *Biometrika*, **85**(1), 1–11.

Gneiting T, Raftery AE (2007). "Strictly Proper Scoring Rules, Prediction, and Estimation." *Journal of the American Statistical Association*, **102**(477), 359–378. doi:10.1198/016214506000001437.

Gonçalves K, Migon HS, Bastos LS (2017). "Dynamic quantile linear model: a Bayesian approach." *arXiv preprint arXiv:1711.00162*.

He X, Pan X, Tan KM, Zhou WX (2023). ***conquer**: Convolution-Type Smoothed Quantile Regression*. R package version 1.3.3, URL https://CRAN.R-project.org/package=conquer.

Huerta G, Jiang W, Tanner MA (2003). "Time series modeling via hierarchical mixtures." *Statistica Sinica*, pp. 1097–1118.

Koenker R (2005). *Quantile regression*. Cambridge University Press, New York.

Koenker R (2025). ***quantreg**: Quantile Regression*. R package version 6.1, URL https://CRAN.R-project.org/package=quantreg.

Kottas A, Krnjajić M (2009). "Bayesian semiparametric modelling in quantile regression." *Scandinavian Journal of Statistics*, **36**(2), 297–319.

Kozumi H, Kobayashi G (2011). "Gibbs sampling methods for Bayesian quantile regression." *Journal of statistical computation and simulation*, **81**(11), 1565–1578.

Kullback S, Leibler RA (1951). "On information and sufficiency." *The annals of mathematical statistics*, **22**(1), 79–86.

Laio F, Tamea S (2007). "Verification Tools for Probabilistic Forecasts of Continuous Hydrological Variables." *Hydrology and Earth System Sciences*, **11**(4), 1267–1277. doi:10.5194/hess-11-1267-2007.

MathWorks (2026). "Working with Quantile Regression Models." Documentation page, accessed 2026-04-19, URL https://www.mathworks.com/help/stats/working-with-quantile-regression-models.html.

Muggeo VMR (2025). ***quantregGrowth**: Non-Crossing Additive Regression Quantiles and Non-Parametric Growth Charts*. R package version 1.7-2, URL https://CRAN.R-project.org/package=quantregGrowth.

Neal RM (2003). "Slice sampling." *The Annals of Statistics*, **31**(3), 705–767. doi:10.1214/aos/1056562461.

Nishimura A, Suchard MA (2023). "Shrinkage with shrunken shoulders: Gibbs sampling shrinkage model posteriors with guaranteed convergence rates." *Bayesian Analysis*, **18**(2), 367–390. doi:10.1214/22-BA1308.

NOAA Physical Sciences Laboratory (2026). "Climate Indices: Monthly Atmospheric and Ocean Time Series." URL https://psl.noaa.gov/data/climateindices/.

Ostwald D, Kirilina E, Starke L, Blankenburg F (2014). "A tutorial on variational Bayes for latent linear stochastic time-series models." *Journal of Mathematical Psychology*, **60**, 1–19.

Ou L, Hunter M, Chow SM, Ji L, Chen M, Hung HJ, Lee J, Li Y, Park J (2021). ***dynr**: Dynamic Models with Regime-Switching*. R package version 0.1.16-2, URL https://CRAN.R-project.org/package=dynr.

Petris G, Gilks W (2018). ***dlm**: Bayesian and Likelihood Analysis of Dynamic Linear Models*. R package version 1.1-5, URL https://CRAN.R-project.org/package=dlm.

Plummer M, Best N, Cowles K, Vines K (2006). "CODA: Convergence Diagnosis and Output Analysis for MCMC." *R News*, **6**(1), 7–11. URL https://journal.r-project.org/archive/.

Prado R, Ferreira MA, West M (2021). *Time Series: Modeling, Computation and Inference*. 2nd edition. Chapman and Hall/CRC Press.

Prado R, Molina F, Huerta G (2006). "Multivariate time series modeling and classification via hierarchical VAR mixtures." *Computational Statistics & Data Analysis*, **51**(3), 1445–1462.

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

Reich BJ, Bondell HD, Wang HJ (2009). "Flexible Bayesian quantile regression for independent and clustered data." *Biostatistics*, **11**(2), 337–352.

Rosenblatt M (1952). "Remarks on a multivariate transformation." *The annals of mathematical statistics*, **23**(3), 470–472.

Sherwood B, Li S, Maidman A (2026). ***rqPen**: Penalized Quantile Regression*. R package version 4.2, URL https://CRAN.R-project.org/package=rqPen.

statsmodels developers (2025). "**statsmodels**: `QuantReg` Documentation." Version 0.14.6 documentation, accessed 2026-04-19, URL https://www.statsmodels.org/stable/generated/statsmodels.regression.quantile_regression.QuantReg.html.

Taddy MA, Kottas A (2010). "A Bayesian nonparametric approach to inference for quantile regression." *Journal of Business & Economic Statistics*, **28**(3), 357–369.

Tokdar S, Cunningham E (2025). ***qrjoint**: Joint Estimation in Linear Quantile Regression*. R package version 2.0-11, URL https://CRAN.R-project.org/package=qrjoint.

US Geological Survey (2016). "National Water Information System data available on the World Wide Web (USGS Water Data for the Nation)." URL http://waterdata.usgs.gov/nwis/.

Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. Fourth edition. Springer, New York. ISBN 0-387-95457-0, URL https://www.stats.ox.ac.uk/pub/MASS4/.

Wang C, Blei DM (2013). "Variational Inference in Non-Conjugate Models." *arXiv preprint arXiv:1209.4360*. URL https://arxiv.org/abs/1209.4360.

West M, Harrison J (1997). *Bayesian Forecasting and Dynamic Models*. 2 edition. Springer, New York. doi:10.1007/b98971.

Yan Y, Zheng X, Kottas A (2025). "A New Family of Error Distributions for Bayesian Quantile Regression." *Bayesian Analysis*. doi:10.1214/25-BA1507.

Yu K, Moyeed RA (2001). "Bayesian quantile regression." *Statistics & Probability Letters*, **54**(4), 437–447.

Affiliation:

Antonio Aguirre and Raquel Barata
Department of Statistics
University of California Santa Cruz
Santa Cruz, CA 95064
E-mail: jaguir26@ucsc.edu, raquel.a.barata@gmail.com

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd